PassRank: Password Guessing Using Adversarial Ranking

Abstract

The understanding of password security heavily depends on the knowledge of the strategies employed by potential adversaries. To this end, modeling guessing attacks has become a pivotal task. Reward models (e.g., RLHF in OpenAI ChatGPT) have been widely applied in the evaluation of natural language generation, and they have become a new paradigm of modern NLP techniques using reinforcement learning.

To better understand password security, this paper provides a *brand new* technical route for password guessing, referred to as PassRank: we, *for the first time*, regard passwords as sequential decision trajectories and design a reward framework tailored for password guessing. We make the learning to rank objectives explicit for the generative password guessing model by designing corresponding rewards. Due to the generality of our approach, our model can be applicable to a wide range of guessing scenarios, i.e., trawling password guessing, targeted password guessing based on personally identifiable information (PII), and conditional password guessing).

Beyond its theoretical importance, this research also holds practical significance. It has the capacity to acquire the password distribution from the leaked password dataset and produce high-caliber guesses. Experiments using 12 large real-world password datasets demonstrate that our adversarial ranking guessing models are effective: (1) PassRank excels in trawling password guessing scenarios, with success rates slightly outperforming competitors at large guess numbers; (2) PassRank-PII for targeted password guessing based on PII, which guesses 58%∼91% of common users within 10^{12} guesses, outperforming its foremost counterpart by 6%∼17%; (3) PassRank-Mask for conditional guessing based on users' partial passwords (e.g., `e***a*rt*`), outperforms state-of-the-art partial guessing models by 7%∼14%. We believe this work takes a significant step in integrating modern reinforcement learning into the realm of password guessing.

1 Introduction

Passwords remain the primary means of user authentication [5, 11, 98] and are crucial for the online daily activities of the approximately 5.2 billion netizens [10]. With the proliferation of online services, the number of web accounts that users need to manage constantly increases, ranging from 80 to 107 [72]. However, users typically can only remember 5-7 different passwords [40]. Unavoidably, users tend to choose popular strings [86], employ personally identifiable information (PII) [87], and are susceptible to side-channel attacks [68, 96] (shoulder surfing [22], keystroke audio feedback [24, 45], etc.). Recently we seem to get accustomed to catastrophic password data breaches from high-profile sites (e.g., 3 billion Yahoo leak [1], 1.1 billion Alibaba credential compromise [94], and 700 million LinkedIn data breach [70]). This upsurge in account usage also exposes users' passwords to a multitude of potential threats. Thus, it is of great importance to thoroughly understand password security.

To this end, modeling password guessing attacks has become a pivotal task. In practice, guess number [12, 66, 80] is found to be a good metric to evaluate password strength, and those easily guessed by an attacker $\mathcal{A}$ are considered to be weak passwords. Therefore, studying password strength from the attacker's perspective is doubly profound. Moreover, the attacker $\mathcal{A}$ can gather extra information (e.g., PII [19, 87], or partial passwords [68]) and create a model from breached passwords to guess and match targeted victims' credentials to steal property and engage in deception.

At IEEE S&P'09, Weir *et al.* [90] proposed the first probabilistic guessing model utilizing a probabilistic context-free grammar (PCFG) [39]. Since then, a series of trawling password guessing approaches that employ users' behavior of choosing popular passwords have been proposed, such as Markov-based [54, 60], LSTM-based [57], GAN-based [33, 68], Cluster-based [47] and AdaMs [67]. Moreover, there is an increase in spell-jacking [3] and credential leaks with PII (e.g., 0.7 billion LinkedIn leaks with PII [2] and 400 million Twitter compromise with PII [4]). In 2015, Wang *et al.* identified these vulnerabilities and subsequently developed a range of targeted password guessing models (e.g., Targeted-Markov [85], TarGuess-I [87] and RFGuess-PII [89]) that employ users' PII to guess targeted victims. Moreover, in the case where an attacker $\mathcal{A}$ obtains partial passwords (e.g., `e***a*rt*`), Pasquini *et al.* [68] proposed the first conditional guessing model, CWAE, followed by PassBERT at USENIX SEC'23 [96] using Bidirectional Encoder Representations from Transformers (BERT) [23]. The threat of attacker $\mathcal{A}$ primarily arises from these scenarios above.

1.1 Motivation and Challenges

Reward models, such as RLHF [18, 26, 44, 63] in OpenAI's ChatGPT, have found widespread application in the field of natural language generation. These innovative models have the unique ability to guide model learning through external reward signals, and this significant development has given rise to a transformative new paradigm in modern NLP techniques.

When it comes to guessing attacks, they typically involve extremely short texts with rich/no semantics. Such attacks can be effectively framed as a learning to rank problem, which suggests a natural fit for an adversarial ranking guessing framework. Moreover, previous models (e.g., Statistics-based [54,90], Random forest-based [89] and FLA-based [57]) have not taken into account the latent space (as highlighted by Pasquini *et al.* [68]). This entails learning password characteristics not by fitting to a given password distribution, but through space transformations using manifolds.

To address these issues, we can design reward models to capture the password generation process at each step, and regard password generation $\mathrm{PW}_{1:T} = (w_0, w_1, ..., w_{T-1})$ as a sequential decision process [71]:

$$\tau = \{s_0, a_0, s_1, a_1, \ldots, s_T, a_T\}. \tag{1}$$

At timestep t, action a_t could be considered as choosing a character from the charset $\mathcal{V}$, with the policy denoted as $\pi_\theta(a_t \mid s_t)$, where the state s_t is the t-gram in previous steps $\mathrm{PW}_{1:(t-1)}$. The probability of a trajectory τ, is given by

$$\begin{aligned} P_\theta(\tau) &= P_\theta(s_0, a_0, s_1, a_1, \cdots, s_T, a_T) \\ &= P_\theta(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P_\theta(s_{t+1} \mid s_t, a_t). \end{aligned} \tag{2}$$

The goal of model is to learn policy $\pi_\theta(a_t \mid s_t)$ such that the expected return is maximized [34, 76]:

$$\begin{aligned} \mathfrak{L}(\theta) &= \mathbb{E}_{\tau \sim p_\theta(\tau)}[\mathcal{T}(\tau)] \\ &= \mathbb{E}_{\mathrm{PW}_{1:T} \sim p_\theta(\mathrm{PW}_{1:T})}[\mathcal{T}(\mathrm{PW}_{1:T})], \end{aligned} \tag{3}$$

where $\mathcal{T}(\mathrm{PW}_{1:T})$ represents the total reward for the password $\mathrm{PW}_{1:T}$. However, how to accurately characterize users' password behavior and obtain the policy π_θ when facing offline attack (e.g., $> 10^9$ as recommended by Wang *et al.* [84]) remains a challenging problem. Here we explain why.

C1: Even minor errors would lead to guess failure. Passwords are extremely short texts with rich semantics. In password guessing, unlike traditional text generation tasks, little changes in passwords will lead to guess failure. For example, when the model generates `passw@rd123` would result in failure when guessing `password123`. This is a more stringent matching than traditional text generation tasks [89].

C2: How to design rewards for password guessing. Password guessing can be regarded as a learning to rank problem, which generates guesses ranking via likelihood in descending order. Fundamentally, a reward signal is an abstraction of the task and serves as the bridge through which we convey the task objective to the agent. Designing rewards in reinforcement learning can be a daunting task, as defining desired behaviors through reward signals can be challenging or may necessitate numerous expert demonstrations. PassGAN [33] and Pasquini *et al.* [68] simply used ResNet [32] as their discriminator, but the true password and the negative password often have little distinction. For example, `123wang` is a real password, while `123yang` is also a real password but is judged as negative by their model [33, 68], which limits the learning capacity for tasks that require the synthesis of rich structures using binary signal. To address this issue, we propose a new ranking model consisting of two parts, the guesser $\mathcal{G}_\theta$ and the ranker $\mathcal{R}_\varphi$ which provides reward signals. How to design reasonable reward signals to convey our learning to rank goals to the guesser $\mathcal{G}_\theta$ remains an unsolved problem.

C3: How to evaluate partial and discrete password state. As a de facto way to implement adversarial imitation learning, applying GANs [30] to generate passwords has some problems. Firstly, GANs are designed for generating real-valued, continuous data but have difficulties in directly generating texts of discrete tokens, such as passwords. Secondly, GANs can provide a score only for a complete password after it has been generated. Measuring the current state with the future score for partially generated passwords can be a non-trivial task once the entire password has been generated.

Roadmap. In Section 2, we present the background and related work in three password guessing scenarios. We elaborate on the design of PassRank in trawling guessing attack, targeted guessing attack, and conditional guessing attack in Section 3, Section 4, and Section 5 respectively. In Section 6, we analyze the performance of PassRank as a password strength meter. Section 7 summarizes this paper.

1.2 Our contributions

In this work, we make the following key contributions:

- **A new technical route.** We, *for the first time*, build a password guessing model using reinforcement learning, treating passwords as trajectories. We employ expert-guided rewards at each timestep during guessing rollouts, facilitating policy gradient optimization. Our training methodology mainly involves adversarial ranking techniques. It learns from the leaked password dataset to generate high-quality guesses.
- **Three improved password guessing models.** By incorporating (pruned) beam search algorithms and enhancing existing PII matching algorithms, we extend our password model to three representative guessing scenarios, namely trawling password guessing, targeted password guessing using PII, and conditional password guessing.

- **Extensive evaluation.** We use 12 large real-world password datasets to demonstrate that our adversarial ranking guessing models are effective: (1) PassRank excels in trawling scenarios, with guessing success rates slightly outperforming competitors at large guess numbers; (2) PassRank-PII for targeted password guessing based on PII, which guesses 58% $\sim$ 91% of common users within 10^{12} guesses, outperforming its foremost counterpart by 6% $\sim$ 17%; (3) PassRank-Mask for conditional guessing based on users' partial passwords, which performs the best among the state-of-the-art models by 7% $\sim$ 14%.
- **Some insights.** When it comes to predicting the next token in the state n-gram patterns, PassRank employs a reward model via reinforcement learning, which converts non-differentiable evaluation function scores and proceeds with gradient updates. This knowledge is valuable for optimizing model training and reducing password generation time by identifying and eliminating password-specific characteristics in the latent space.

2 Background and related work

2.1 Three guessing scenarios

Trawling password guessing. Trawling guessing means that the attacker $\mathcal{A}$ does not care who the specific target is, and its only goal is to guess more passwords under the guess number allowed (e.g., $> 10^9$ as recommended by Wang *et al.* [84]).

At IEEE S&P'09, Weir *et al.* [90] proposed the first password guessing models based on probabilistic context-free grammar (PCFG) [39]. The core assumption [90] of the model is the segments of letters, numbers, and special symbols in a password are independent of each other. For example, the password `wang123!` is composed of a $\mathcal{L}_4$ segment `wang`, a $\mathcal{D}_3$ segment `123`, and a $\mathcal{S}_1$ special segment `!`, with a template of $\mathcal{L}_4\mathcal{D}_3\mathcal{S}_1$. The password generation process can be modeled using a template rule $(\mathcal{R} \rightarrow \mathcal{L}_4\mathcal{D}_3\mathcal{S}_1)$ and segment rules $(\mathcal{L}_4 \rightarrow \texttt{wang}, \mathcal{D}_3 \rightarrow \texttt{123}, \mathcal{S}_1 \rightarrow \texttt{!})$. Therefore, the PCFG models `wang123!` as $P(\texttt{wang123!}) = P(\mathcal{L}_4\mathcal{D}_3\mathcal{S}_1) \times P(\mathcal{L}_4 \rightarrow \texttt{wang}) \times P(\mathcal{D}_3 \rightarrow \texttt{123}) \times P(\mathcal{S}_1 \rightarrow \texttt{!})$. In 2014, Ma *et al.* [54] implied that the PCFG algorithm is statistics-based models that crack passwords by counting the frequency of elements in the training set (e.g., the $\mathcal{LDS}$ segments in PCFG), which have the inherent limitations of password sparseness and overfitting [54]. Houshmand *et al.* [36] introduced keyboard $\mathcal{K}$ and phrase $\mathcal{P}$ structures to introduce semantics between passwords. In 2014, Veras *et al.* [83] proposed the addition of semantic analysis to $\mathcal{L}$-segment, with semantic-PCFG.

In 2005, Narayanan and Shmatikov [60] introduced the first autoregressive password guessing algorithm by utilizing the Markov chain [38, 74] to capture dependencies between passwords. The use of Markov models helps reduce the password search space, thereby enhancing the rate of guessing attacks within memory constraints. Additionally, passwords that adhere to human memorability patterns often exhibit Markov properties. At USENIX'16, Melicher *et al.* [57] pioneered the incorporation of deep learning into password guessing. Unlike fixed-order Markov models [60], they employed neural networks to learn relationships between characters. Specifically, they constructed a neural network comprising LSTMs [35], denoted as FLA (Fast, Lean, and Accurate).

In 2019, Hitaj *et al.* [33] proposed PassGAN, a method that learns password samples between the real and generated passwords to simulate password distribution. They use a generative neural network to capture password features and a discriminative neural network [15, 30] to assess the similarity between training and generated ones. The goal is to make it difficult for the discriminative network to distinguish between them. At this point, it indicates that the generator has the capability to generate real passwords. After that, Pasquini *et al.* [68] alleviated the mode collapse problem of training GANs and Wasserstein autoencoder via WGAN and CWAE [1], so that the cracking rate of GANs-based approaches under large guesses has been significantly improved. On this basis, they constructed two password guessing frameworks, that is, Conditional Password Guessing (CPG) and Dynamic Password Guessing (DPG). However, both PassGAN [33] and Pasquini *et al.* [68] are not probabilistic models, which simply utilize ResNet [32] as their generator and discriminator without accounting for the cyclic dependencies within passwords, resulting in unsatisfactory performance (see Table 2).

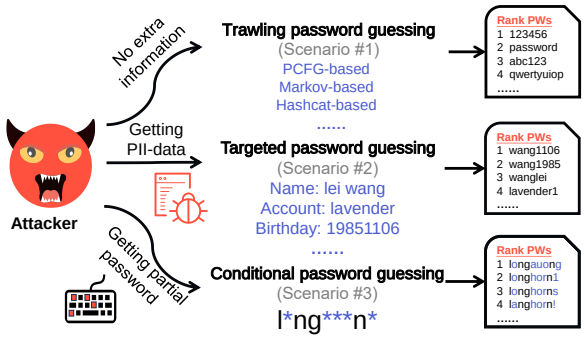


Figure 1: The illustration of three real-world attack examples, where attacker $\mathcal{A}$ collects victims-relevant information or not to launch specific password guessing attacks.

Targeted password guessing. There are many kinds of PII about users, such as demographic related information (name, birthday, age, phone number, education, gender, etc.). The goal of targeted password guessing is to crack the password of a given user in a given PII as quickly as possible [87].

In 2015, Wang *et al.* [85] introduced the first targeted probabilistic model known as TarMarkov. Grounded in the principles of Markov [60], this approach was designed to leverage the frequency of names within a training password set in order

[1] More specifically, Pasquini *et al.* [68] maintain the core GANs and Autoencoders framework mostly unaltered due to the exceptional performance of the gradient-penalty Wasserstein GAN and Context Wasserstein Autoencoder, abbreviated as WGAN and CWAE.

Table 1: A concise summary of labeled datasets used in three real-world attack scenarios, in which the primary aim of all attack models is to effectively forecast their corresponding outputs (Y) based on the provided inputs (X) and provided explanatory examples of these mapping relationships. These are also the primary focus of the three scenarios in this work.

Attack scenarios	Explanations of attack	Labeled sets Inputs X, accompanied by its outputs Y	Example $(x \rightarrow y)$ with $(x \in X, y \in Y)$
Trawling guessing	The attacker doesn't care who the specific target is, and its only goal is to guess as more passwords as possible [84].	Passwords with the begin symbol B_s with them shifted right with the end symbol E_s.	B_s `password` $\rightarrow$ `password` E_s
Targeted guessing	Utilizing models to inference password with provided specific users' PII [87].	PII (email, user name, name, birthday, phone, etc.) with corresponding passwords	(`wl@gmail.com`, `leiwang`, `lavender`, `19851106`, `123-456-7890`)$\rightarrow$`wang1106`
Conditional guessing	Complete password filling the pivot [68].	Partial passwords with complete passwords.	`P***w0rd****` $\rightarrow$ `P@ssw0rd123!`

to estimate the likelihood of specific users choosing name-based passwords. In 2016, Li *et al.* [48] proposed a targeted guessing model based on PCFG [90]. After that, Wang *et al.* [87] showed that their TarGuess-I is more efficient than previous models using users' PII to crack users' passwords, which can gain success rates over 20% with just 100 guesses. TarGuess-I adds 6 PII tags (name, username, birthday, phone number, etc.) to the three basic tags of $\mathcal{LDS}$ in PCFGs [39].

Conditional password guessing. Conditional password guessing means that an attacker $\mathcal{A}$ has obtained an incomplete password (e.g., some characters) of the victim by exploiting various side channel attacks [22, 24, 45, 68, 96] (e.g., shoulder surfing, keystroke audio feedback, and thermal residue), then initiates a password guessing attack (e.g., `e***a*rt*`, `p***ord**3`). A plain way to tackle conditional password guessing is by using trawling models to create numerous potential passwords [68]. However, the task of efficiently removing irrelevant passwords from this pool can be resource-intensive and demand significant storage capacity.

At IEEE S&P'21, Pasquini *et al.* [68] proposed the first deep conditional password guessing model based on Wasserstein Autoencoder [79]. They refer to the final model as context wasserstein autoencoder (CWAE) which consists of an encoder embedding a partial password into latent representations, and a decoder converting the latent representations of the partial password into passwords. In 2023, Xu *et al.* [96] proposed a state of the art approach for conditional password guessing based on bi-directional encoder representation from transformers (BERT) [23] using pre-train and fine-tuing paradigm [52]. However, PassBERT [96] requires a massive pretraining password to achieve results competitive with CWAE [68], which itself represents a biased advantage over other models. Additionally, BERT [23] was originally designed to address issues of long-range dependencies and parallel processing between words, which are not pertinent concerns in password guessing. Therefore, choosing Transformer [82] models like BERT family [23] as the backbone network may not be a suitable option for trawling guessing.

Summary. We summarized the training sets, denoted as X, and the generated guesses Y for the three password guessing scenarios in Table 1. Moreover, in Figure 1, we provided an overview of the information held by each attacker $\mathcal{A}$. In the trawling scenario, there is no additional information available to the attacker $\mathcal{A}$. In the targeted scenario, the attacker possesses the victim's password using PII. In conditional scenario, the attacker has access to the victim's partial password.

Table 2: Guessing performance of our PassRank in comparison with other adversarial generative models (i.e., PassGAN [33] and WGAN [68]) training on 50% Rockyou to crack the remaining 50% Rockyou at large guess number.

Guess number	PassGAN [33]	WGAN [68]	PassRank
$1 \cdot 10^8$	7.62%	9.12%	**54.01%**
$1 \cdot 10^9$	18.54%	23.95%	**62.38%**
$1 \cdot 10^{10}$	29.28%	36.51%	**71.99%**
$5 \cdot 10^{10}$	38.03%	47.91%	**75.69%**

2.2 Probabilistic generative models

Probabilistic generative models [8, 59] are a significant class of models in the fields of machine learning, which are widely used in the field of password guessing [33, 57, 68]. They refer to a set of models used for randomly [30, 42] or conditionally [15] generating observable passwords from latent space.

Theoretical guarantee of generative models. Assuming there exists a random vector $\mathbf{x}$ in a discrete high-dimensional space $\mathcal{X}$, following a distribution $P(\mathbf{x}), \mathbf{x} \in \mathcal{X}$, generative models aim to learn a parameterized model $P_\theta(\mathbf{x})$ based on breach passwords $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \cdots, \mathbf{x}^{(N)}$. This model approximates the unknown distribution $P_r(\mathbf{x})$ and can be used to generate passwords that closely resemble *real* passwords. Generative models typically serve two fundamental purposes: density estimation and sample generation (i.e., sampling [16, 62]).

As previously mentioned in Section 2.1 password guessing generative models, such as CWAE [68], explicitly construct the density function $P(\mathbf{x};\theta)$ for generating passwords and then use maximum likelihood estimation to estimate the parameters, known as *explicit density models*. Taking CWAE [68] as an example, the density function of the CWAE is represented as $P(\mathbf{x},\mathbf{z};\theta) = P(\mathbf{x} \mid \mathbf{z};\theta)P(\mathbf{z};\theta)$. Although neural networks are employed to estimate $P(\mathbf{x} \mid \mathbf{z};\theta)$, it is assumed to belong to a parameterized distribution family, with the network being used to predict the parameters of the distribution. This limits the capabilities of neural networks.

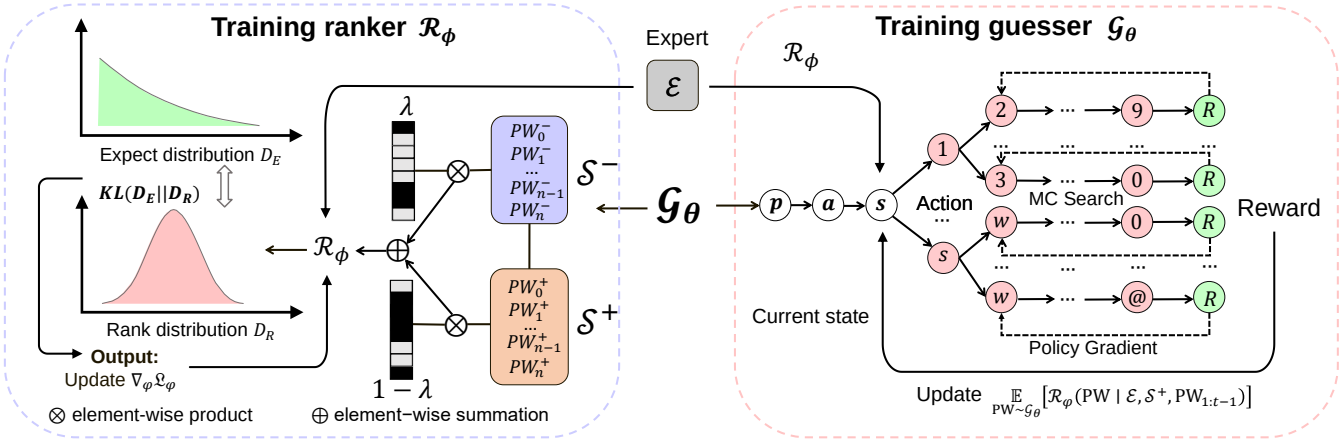


Figure 2: Our PassRank and training phases primarily divided into two parts: ranker $\mathcal{R}_\varphi$ and guesser $\mathcal{G}_\theta$. The training of ranker $\mathcal{R}_\varphi$, is to assign corresponding rankings to mixture passwords under the guidance of expert $\mathcal{E}$, which represents the Top-1w passwords from the training set serving as the reference for imitation. $\mathcal{S}$+ consists of n randomly samples from the training set, $\mathcal{S}$- comprises n passwords generated by $\mathcal{R}_\theta$. By mixing these sets with blending ratios ranging from 0 to 1, we create n mixup password sets. The expected distribution is expected to be monotonically decreasing due to the changing blending ratios. Meanwhile, the training of the guesser $\mathcal{G}_\theta$ generates a password with a high reward passing back to the intermediate action value via Monte Carlo tree search [14]. Thus, it can generate passwords similar to those generated by the expert $\mathcal{E}$. There's a mutual adversarial relationship between the guesser $\mathcal{G}_\theta$ and the ranker $\mathcal{R}_\varphi$.

Implicit density models. However, if the goal is merely to have a model capable of generating samples that adhere to the password distribution $P_r(\mathbf{x})$, there is *no need* to explicitly estimate the density function of the password distribution [8, 59]. Assuming there is a simple and *easy-to-sample* distribution $P(\mathbf{z})$ in a low-dimensional space $\mathcal{Z}$ (e.g., often chosen as the standard multivariate normal distribution $\mathcal{N}(\mathbf{0}, \mathbf{I})$), a neural network is employed to learn a mapping function $\mathcal{M} : \mathbf{z} \to \mathbf{x}$, known as the *generative network*. Leveraging the powerful fitting capabilities of neural networks [68], this ensures that $\mathcal{M}(\mathbf{z})$ follows the distribution $P_r(\mathbf{x})$. Such models are referred to as *implicit density models*. The term *implicit* signifies that it does not explicitly model $P_r(\mathbf{x})$ but instead models the generation process. A key aspect of implicit density models ensure that the samples generated by the generative network indeed follow the true password distribution. Since we do not construct an explicit density function, traditional methods like maximum likelihood estimation cannot be used for training [97]. Generative adversarial networks (GANs) [15, 30] provide a solution by employing an adversarial training approach to ensure that the generated samples conform to the genuine password distribution, as the work of Hitaj *et al* [33].

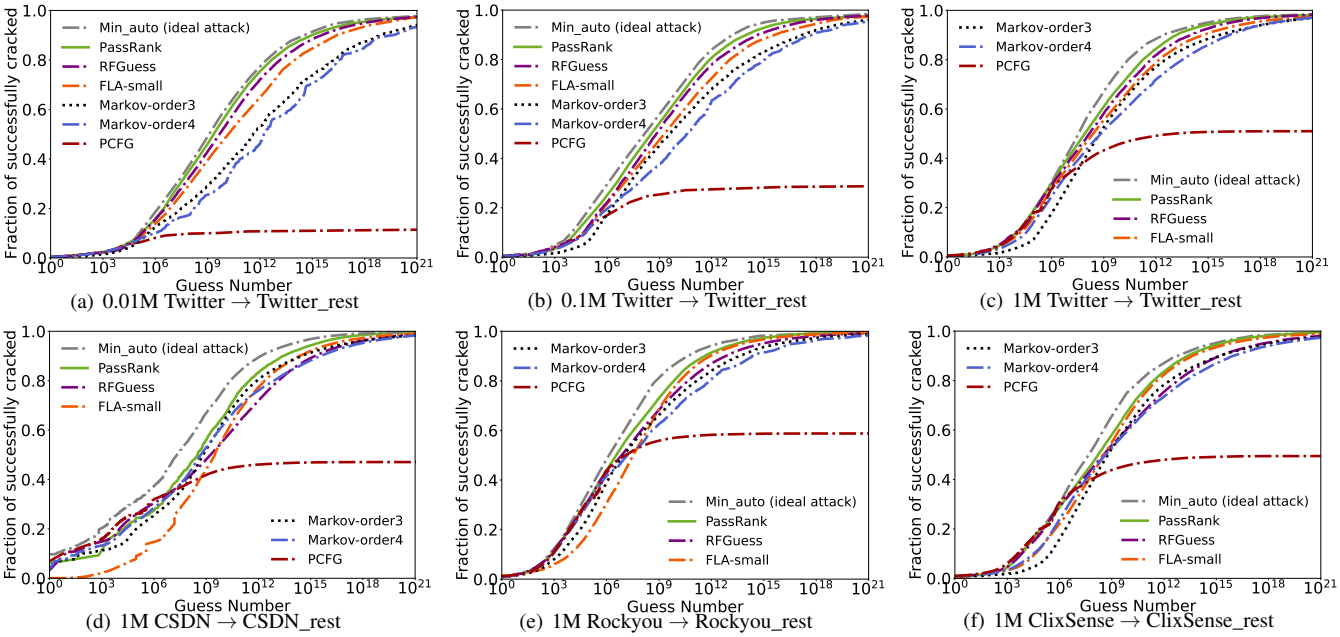
Password generation as sequential decisions. Password generation can naturally be seen as a sequence of choices made by individuals on a keyboard. Reward models are widely used to assess sequence generation in large language modelings(e.g., RLHF [56, 75], using given prompts to get the reward from human evaluation). There are lots of sequence generation models [49, 97], but they cannot be directly applied to the task of password guessing, as we explained in Section 1.1. Passwords can also be regarded as sequential decisions with Markov property. The goal of the model is to learn a policy $\pi_\theta(a \mid s_t)$ such that the expected return $\mathbb{E}_{\tau\sim p_\theta(\tau)}[\mathcal{T}(\tau)]$ of generating password is maximized. The reward model becomes more precise, resulting better match password guessing.

2.3 Generative adversarial imitation learning

Password guessing is a learning to rank process that consistently prioritizes high-likelihood passwords for attacks, as explained in Section 2.1. We can consider this process as an expert strategy $\mathcal{E}$, and our goal is to train an agent capable of acquiring the skill of password guessing. This naturally constitutes an imitation learning problem [31, 43, 61, 73, 76], which is a more effective strategy for a sequential generation with near-optimal cost-to-go oracles [76]. The essence of generative adversarial imitation learning (GAIL) is to imitate the strategy of the expert $\mathcal{E}$ in the environment. We consider the interaction with the expert $\mathcal{E}$ to signals when training differently with Jonathan *el al.* [76]. The strategy, denoted as π, acts like a generator in a generative adversarial network. When given a state s, it produces an action a for that state. The discriminator takes the state-action pair (s_t, a_t) as input and outputs a value indicating its belief in whether this pair is from the agent's strategy or the expert's $\mathcal{E}$. Therefore, the agent is increasingly similar to that strategy of the expert $\mathcal{E}$.

3 Trawling password guessing

Trawling password guessing [89], as one of the most traditional and common attack scenarios, involves attempting password guesses without using any personally identifiable information (PII) or extra information. It is currently one of the most basic but important threats to password security.



(a) 0.01M Twitter → Twitter_rest (b) 0.1M Twitter → Twitter_rest (c) 1M Twitter → Twitter_rest

(d) 1M CSDN → CSDN_rest (e) 1M Rockyou → Rockyou_rest (f) 1M ClixSense → ClixSense_rest

Figure 3: Guessing performance of our PassRank in comparison with other approaches (i.e., RFGuess [89], PCFG [90], 3/4-order Markov [54, 60], FLA [57]) in the intra-site guessing scenarios with all 1M (million) and adaptive training sets. Dataset_rest means the original dataset excluding the corresponding training set. Min_auto [81] is an idealized strategy where a password is considered cracked if any of these six real-world password models succeed. Our PassRank performs better than its counterparts.

3.1 Password guessing modeling

Suppose the password charset in this work is denoted by $\mathcal{V}$, and at time step t, the previous tokens generated in the sequence are $\mathrm{PW}_{1:t} = (w_0, w_1, ..., w_{t-1})$, where all tokens $w_i \in \mathcal{V}$. For sequence tasks with varying lengths, a recurrent neural network (RNN) is a commonly used neural network structure. To mitigate gradient issues, LSTMs [35] were introduced, enhancing the capture of semantic relationships within long sequences compared to classical RNNs. Gated recurrent unit (GRU) [17], with a simpler structure but competitive to LSTMs [35], is chosen as the basis for our neural network.

Our PassRank and training phases are primarily divided into two parts: training ranker $\mathcal{R}_\varphi$ and guesser $\mathcal{G}_\theta$. The training of ranker $\mathcal{R}_\varphi$, is to assign corresponding rankings to mixture passwords under the guidance of expert $\mathcal{E}$, which represents the Top-1w passwords from the training set serving as the reference for imitation. However, the training of the guesser $\mathcal{G}_\theta$ generates a password with a high reward passing to the intermediate action value via Monte Carlo Tree search [14]. Thus, it can generate passwords similar to those generated by the expert $\mathcal{E}$. There's a mutual adversarial relationship between guesser $\mathcal{G}_\theta$ and ranker $\mathcal{R}_\varphi$. Although the description is sequential, the actual occurs concurrently.

To prototype the model discussed, we use GRU [17] as the guesser model, which maps the input embedding representations $(\mathbf{e}_0, \ldots, \mathbf{e}_{T-1})$ of the sequence $\mathrm{PW}_{1:T} = (w_0, \ldots, w_{T-1})$ into a sequence of hidden states $(\mathbf{h}_0, \ldots, \mathbf{h}_{T-1})$ by using the update function $\mathbf{g}$ recursively,

$$\mathbf{h}_t = \mathbf{g}(\mathbf{h}_{t-1}, \mathbf{e}_t). \tag{4}$$

Moreover, a softmax output layer $\mathbf{z}$ maps the hidden states into the output action distribution taken by the guesser $\mathcal{G}_\theta$,

$$\mathbf{z}(\mathbf{h}_t) = \mathrm{softmax}(\mathbf{c} + \mathbf{V}\mathbf{h}_t), \tag{5}$$

where $\mathbf{c}$ and $\mathbf{V}$ are bias vector and weight matrix. To deal with the common vanishing and exploding gradient [29] of the backpropagation through time, we leverage the GRU cells [17] to implement the update $\mathbf{g}$ in Equation. (4).

PassGAN [33] employs a ResNet [32] as the backbone network [2], while the discriminator employs a multi-layer perceptron to output a binary probability distribution for determining whether a password originates from passwords or fake ones. However, Our PassRank consists of a password guesser $\mathcal{G}_\theta$ and ranker $\mathcal{R}_\varphi$, where $\mathcal{R}_\varphi$ assigns relative rankings to passwords when provided with a ranking expert $\mathcal{E}$:

- The learning objective of the guesser $\mathcal{G}_\theta$, is to have ranker $\mathcal{R}_\varphi$, assign a higher rank to the generated sample $\mathcal{S}^-$ compared to the real passwords $\mathcal{S}^+$.
- The learning objective of the ranker $\mathcal{R}_\varphi$, is to assign a lower rank to the generated sample $\mathcal{S}^-$ compared to the real passwords $\mathcal{S}^+$.

In this assumption, the ranker $\mathcal{R}_\varphi$ continuously scores the guesser $\mathcal{G}_\theta$ under the guidance of expert $\mathcal{E}$ to enhance the

[2] https://github.com/brannondorsey/PassGAN

guesser's generative capability. Consequently, this can be regarded as a process where $\mathcal{G}_\theta$ and $\mathcal{R}_\varphi$ engage in a minimax game. The proposed password ranker $\mathcal{R}_\varphi$, shares a similar recurrent neural network structure. Firstly, it maps the connected password matrix into embedded feature vectors $\mathbf{y}_s$ through a series of nonlinear functions $\mathfrak{F}$. Subsequently, ranking scores for the password features $\mathbf{y}_s$ are computed using expert features $\mathbf{y}_e$ extracted in advance by $\mathcal{R}_\varphi$.

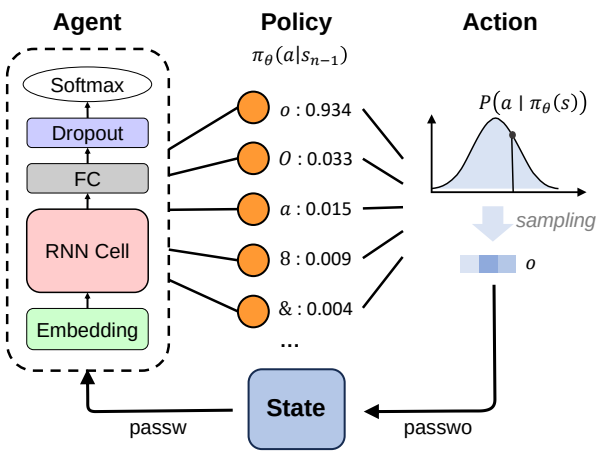


Figure 4: The sequential decision process of our model. It consists of an agent that learns a policy $\pi_\theta(s)$ to predict the next token in a given partial password state by taking actions.

Ranking scores. Taking inspiration from the ranking steps commonly used in network search algorithms like PageRank [64], we introduce a similarity metric capable of assessing relative ranks between passwords where we formulate the relevance cosine score of an input PW concerning a given expert $\mathcal{E}$ and comparison passwords $\mathcal{S}$ as follows:

$$P(\mathrm{PW}|\mathcal{E},\mathcal{S}) = \frac{\exp\left[\gamma\cos(\mathbf{y}_{\mathrm{PW}},\mathbf{y}_e)\right]}{\sum_{\mathrm{PW}'\in\mathcal{S}\bigcup\{\mathrm{PW}\}}\exp\left[\gamma\cos(\mathbf{y}_{\mathrm{PW}'},\mathbf{y}_e)\right]}, \quad (6)$$

Here, $\mathbf{y}_{\mathrm{PW}}$ and $\mathbf{y}_e$ are the feature vectors of the input and expert password. For a set, the ranking score is the average of each individual over the expert $\mathcal{E}$, denoted as $\mathcal{R}_\varphi(\mathrm{PW}|\mathcal{E},\mathcal{S})$.

3.2 Model training and password generation

Training guesser. Suppose $\mathrm{PW}_{1:t-1} = (w_0, w_1, ..., w_{t-1})$ is the current state, the next token w_t that selected in the next action a_t from the policy $\pi_\theta(w_t|\mathrm{PW}_{1:t-1})$. Once the generator reaches the end (i.e., E_s), it receives a ranking reward $\mathcal{T}(\mathrm{PW}|\mathcal{E},\mathcal{S})$ according to the comparison password $\mathcal{S}$ and its related expert $\mathcal{E}$ as we illustrated at Figure 2.

Note that in traditional reinforcement learning, the current reward is compromised by the rewards from intermediate states and future states. However, in password guessing, the guesser $\mathcal{G}_\theta$ obtains the reward if and only if one password has been completely generated, which means no intermediate reward is gained before the sequence hits the end symbol. To relieve this problem, we utilize the Monte Carlo rollouts methods [13, 49, 58, 97] to simulate intermediate rewards:

$$\mathcal{T}_{\theta,\varphi}(\mathrm{PW}_{1:t-1},\mathcal{E}) = \mathop{\mathbb{E}}_{\mathrm{PW}\sim\mathcal{G}_\theta}\left[\mathcal{R}_\varphi(\mathrm{PW}|\mathcal{E},\mathcal{S}^+,\mathrm{PW}_{1:t-1})\right], \quad (7)$$

Here, PW represents the complete password sampled by rollout methods with sequence $s_{1:t-1}$. To be more specific, the beginning tokens $(w_0, w_1, ..., w_{t-1})$ are fixed, and the rest tokens are consecutively sampled by $\mathcal{G}_\theta$ until the last token w_T is generated. Assuming we conduct N trials and obtain N trajectories, denoted as $\tau^{(1)}, \tau^{(2)}, \ldots, \tau^{(N)}$, with corresponding total returns of $\mathcal{T}_{\theta,\varphi}(\tau^{(1)}), \mathcal{T}_{\theta,\varphi}(\tau^{(2)}), \ldots, \mathcal{T}_{\theta,\varphi}(\tau^{(N)})$, the total return can be approximated as follows:

$$\hat{\mathcal{T}}_{\theta,\varphi}(\mathrm{PW}_{1:t-1},\mathcal{E}) = \frac{1}{N}\sum_{n=1}^{N}\mathcal{T}_{\theta,\varphi}(\tau^{(n)}), \quad (8)$$

As $N \to \infty$, the empirical estimator $\hat{\mathcal{T}}_{\theta,\varphi}(\mathrm{PW}_{1:t-1},\mathcal{E})$ converges in probability to the true value $\mathcal{T}_{\theta,\varphi}(\mathrm{PW}_{1:t-1},\mathcal{E})$, (as demonstrated in Appendix B.2) which denoted as δ_t.

Policy gradient. With the feasible intermediate rewards, we can finalize the objective function for a complete password. Refer to the proof in Appendix B.1, the gradient of the objective function for guesser $\mathcal{G}_\theta$ can be formulated as:

$$\nabla_\theta\mathfrak{L}_\theta = \mathop{\mathbb{E}}_{\mathrm{PW}_{1:T}\sim\mathcal{G}_\theta}\left[\sum_{t=1}^{T}\sum_{w_t\in\mathcal{V}}\nabla_\theta\delta_t\log\pi_\theta(w_t|\mathrm{PW}_{1:t-1})\right], \quad (9)$$

where ∇_θ is the partial differential operator. $\mathbb{E}_{s_{1:T}\sim\mathcal{G}_\theta}$ is the mean over all sampled complete password based on current guesser's parameter θ within one batch. Note that we only compute the partial derivatives for θ, as the $\mathcal{R}_\varphi$ is fixed during the training of the guesser. Importantly, different from the policy gradients methods in other works [33, 68, 97], our method replaces the simple binary outputs with a ranking system, which can better reflect the quality of the imitate password and facilitate effective training of the generator $\mathcal{G}_\theta$.

Training ranker. To train the ranker's parameters set φ, we keep the θ parameters fixed. We generate samples $\mathcal{S}^-$ using the guesser $\mathcal{G}_\theta$ and sample the same passwords from the ground truth as $\mathcal{S}^+ \sim \mathcal{P}_h$ in $\mathcal{D}$. Next, we create mixup samples $\mathcal{S}_i^M = \lambda_i\mathcal{S}^+ + (1-\lambda_i)\mathcal{S}^-$, where $\lambda_i \sim \mathcal{U}(N)$. Then, we obtain the rank distribution $\mathcal{T}_{\theta,\varphi}(\mathcal{S}_i^M,\mathcal{E})$. We update φ by measuring the difference between the rank distribution and the true λ_i distribution using Kullback–Leibler divergence:

$$\nabla_\varphi\mathfrak{L}_\varphi = -\sum_{i=0}^{T-1}\nabla_\varphi\mathcal{T}_{\theta,\varphi}(\mathcal{S}_i^M,\mathcal{E})\log\lambda_i. \quad (10)$$

In all, we have summarized the complete and detailed training algorithm in the Appendix A.1 and Algorithm 1.

Password generation. As outlined in Appendix A.1 and Algorithm. 2, to align with the length distribution of actual passwords, we have developed an adaptive password generation

Table 3: Basic information regarding our 12 password datasets used in our experiments, including details on the web service, language, when leaked, unique passwords after filtering, with PII or not, and the size of those branch password datasets.†

Dataset	Web service	Language	When leaked	Total PWs	Length>30	Removed %	Unique PWs	With PII
Taobao	E-commerce	Chinese	Feb., 2016	15,072,418	88	0.01%	11,633,759	
Dodonew	E-commerce	Chinese	Dec., 2011	16,283,140	13,4758	0.15%	10,135,260	
CSDN	Programmer	Chinese	Dec., 2011	6,428,632	0	0.01%	4,037,605	
Wishbone	Social forum	English	Jan., 2020	10,092,037	250	0.01%	5,933,902	
Mate1	Dating website	English	Mar., 2016	27,401,505	12,430	0.06%	11,916,080	
000Webhost	Web hosting	English	Oct., 2015	15,299,907	4,159	0.76%	10,526,769	
Twitter	Social forum	English	June,2016	40,669,963	282,149	0.69%	10,583,709	
LinkedIn	Job hunting	English	Jan., 2012	54,656,615	17,162	0.22%	34,282,741	
Rockyou	Social forum	English	Dec., 2009	32,603,387	3,140	0.07%	14,326,970	
12306	Train ticketing	Chinese	Dec., 2014	129,303	129,303	0	117,808	✓
ClixSense	Paid task platform	English	Sep., 2016	2,222,045	0	0	1,628,018	✓
Rootkit	Hacker forum	English	Feb., 2011	69,330	5	0.01%	56,835	✓

†PWs represents passwords, and PII for personally identifiable information. We filter out passwords exceeding 30 in length or contain non-ASCII characters.

algorithm. Initially, we start with an empty stack $\mathcal{Q}$ for prefixes and a lookup table LUT for probability. The algorithm iterates by popping a prefix from the stack and predicting token probabilities with our guesser $\mathcal{G}_\theta$. If this probability exceeds the threshold or the token is E_s, then the prefix is a valid password and is added to the set $\mathcal{P}$. Otherwise, the prefix is extended with the token, its probability is updated in LUT, and it's pushed back onto the stack for popping. This recursive process continues until the stack is empty. Ultimately, the algorithm returns the set of generated passwords $\mathcal{P}$.

Implementation details. The experiments were conducted on a workstation operating on Ubuntu 22.04 LTS. The models utilized in this study were implemented using PyTorch [3] and TensorFlow[4] with their corresponding libraries. Notably, the experimental machine was equipped with a Xeon Gold 6230R @2.1GHz CPU, 256G Memory, NVIDIA RTX 3090 GPU (with 24GB of VRAM), and a 4TB hard drive. We posit that this configuration is attainable for practical attackers $\mathcal{A}$.

3.3 Experimental setup and results

Datasets. We evaluate the existing password guessing approaches and our PassRank model based on 12 large real-world password datasets (see Table 3), a total of 421.79 million(M) passwords. Eight of our password datasets are from English sites and four are from Chinese ones. We have observed that the original datasets include abnormal passwords that do not seem to be user-chosen passwords or relevant information, either because they are too long (exceeding 30 tokens) or too short (less than 4 tokens) the same as precious work [57, 68, 89, 90]. Therefore, we have initiated the task of cleaning the datasets before conducting any experiments. Our first step involves removing passwords consisting of symbols beyond the 95 printable ASCII tokens (i.e., Table 10).

Ethical considerations. While the datasets in our research have been widely used in lots of literature [9, 65, 68, 87, 96], some of them remain private and contain sensitive information (see Table 4). To ensure the privacy of each individual account, we have only reported aggregated statistics. Besides, we store and process these datasets on computers not linked to the Internet. Our use of these datasets has received approval from our center's Institutional Review Board (IRB) and has been vetted by privacy experts. Furthermore, our use of these datasets is not only beneficial for password guessing but also for admins to protect user account security, which is consistent with security research best practices (see, e.g., [9, 68]).

Experimental setup. We compare PassRank with four leading guessing model (i.e., PCFG [90], Markov [54], FLA [57], and RFGuess [89]). Also, we introduce the Min_auto approach [81] to avoid the bias of a single approach as the best voting for all models. The setup of them are as follows:

- **PCFG**. The PCFG we use is consistent with [54], that is, the probability of $\mathcal{L}$ segment comes from the training set, which surpasses the original PCFG, Weir *et al.* [90].
- **Markov**. For Markov, due to the great influence of order, this paper carries out both 3-order and 4-order experiments at the same time and adopts the Laplace smoothing and end symbol regularization as Ma *et al.* [54].
- **FLA**. We use the source code[1] of FLA [57], and follow its recommended parameters in our experiments. More specifically, we train a model consisting of three LSTM layers with 200 cells (i.e., *small* model in [57]) in each layer and two fully connected layers, a total of 20 epochs.
- **RFGuess**. The RFGuess we use is consistent with Wang *et al.* [89], and we train the random forest with 30 decision trees. Meanwhile, the minimum number of leaf nodes is 10, and the maximum ratio of features is 80%.
- **PassRank**. As detailed in Appendix A.1, we train the PassRank with $\gamma = 0.1$. The loss function utilized is KL divergence, and the optimizer is Adam [41], with an initial learning rate of 0.001 and betas of (0.5, 0.999) for momentum. We train the model with 5 warm up epochs and a total of 20 epochs. More specifically, we train the guesser $\mathcal{G}_\theta$ with 17 group numbers and 0.3 dropout rate. Meanwhile, we train ranker $\mathcal{R}_\varphi$ with a 0.3 dropout rate.
- **Min_auto**. It is an idealized guessing approach [81], in which a password is considered cracked as soon as any of these real-world guessing models cracks it.

[3] https://pytorch.org/
[4] https://www.tensorflow.org/

[1] https://github.com/cupslab/neural_network_cracking

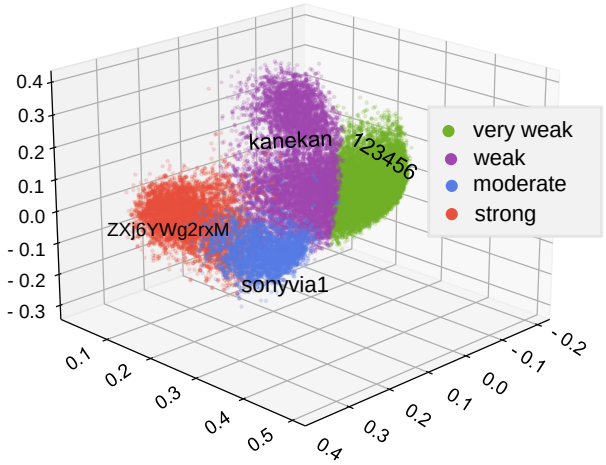


Figure 5: The PCA-based visualization and K-means [55] clustering of ranker $\mathcal{R}_\theta$ latent space for passwords. Green represents weaker passwords, such as `123456`, followed by purple, blue, and red, demonstrating ranker sorting capability.

Experimental results. Given the computationally intensive nature of enumerating substantial guesses, we employ the Monte-Carlo algorithm [21, 51] to approximate the number of attempts needed for an attacker $\mathcal{A}$ to discern a password through guesses made in decreasing order of likelihood.

Our experiment is mainly divided into two parts: exploring the impact of the password training dataset on various models and the differences in performance on commonly used password training datasets. Figure 3 shows the results. In our cases, our PassRank performs slightly better than FLA [57], and beats PCFG [90] and Markov [54] beginning at 10^6 guesses.

We conduct an apples-to-apples performance comparison of these approaches using an adaptive training dataset size, which includes samples from Twitter with sizes of 0.01 M(=Million), 0.1 M, and 1 M. PassRank achieved results of 90.23%, 94.09%, and 95.17%, respectively, when guessing numbers at 10^{15}. It is evident that as the training dataset size increases, the guessing ability of each model also improves. This is especially noticeable in the case of PCFG [90], which exhibits a significant dependency on sample size (9.11% $\sim$ 48.03%). In four additional experiments with different 1M training sets, PassRank achieves the highest crack rate at large guess number (i.e., $>10^7$) as shown in Figure 3.

In addition, we visualized the latent space of the ranker $\mathcal{R}_\varphi$ for complete passwords, reducing the dimensionality from 512 dimensions to 3 dimensions using PCA. We also employed the K-means [55] clustering method along with the Elbow algorithm to select an appropriate number of clusters (i.e., 4) with selected passwords, as shown in Figure 5. The `123456` is the weakest, so it appears in the green category. Next is `kanekan` in the purple category. In the red category are the strongest passwords (e.g., `ZXj6YWg2rxM`). It is evident that the ranker $\mathcal{R}_\theta$ exhibits a clear capability to rank passwords, as mentioned in Section 1.1.

Table 4: Basic information and description of our PII datasets.

Dataset	Language	Items num	The types of PII attributes
PII-ClixSense	English	2,222,045	Email, User name, Name, Birthday
PII-000Webhost	English	79,580	Email, User name, Name, Birthday
PII-Rootkit	English	69,418	Email, User name, Name, Birthday
PII-12306	Chinese	129,303	Email, User name, Name, Birthday, Phone
PII-CSDN	Chinese	77,216	Email, User name, Name, Birthday, Phone
PII-Dodonew	Chinese	161,517	Email, User name, Name, Birthday, Phone

4 Targeted password guessing

There are many kinds of PII about users, such as demographic related information (name, birthday, age, occupation, education, gender, etc.). The goal of targeted password guessing is to crack the password of a given user in a given service as quickly as possible [87]. Targeted password guessing predicts the probability of the real passwords (PWs) given PII:

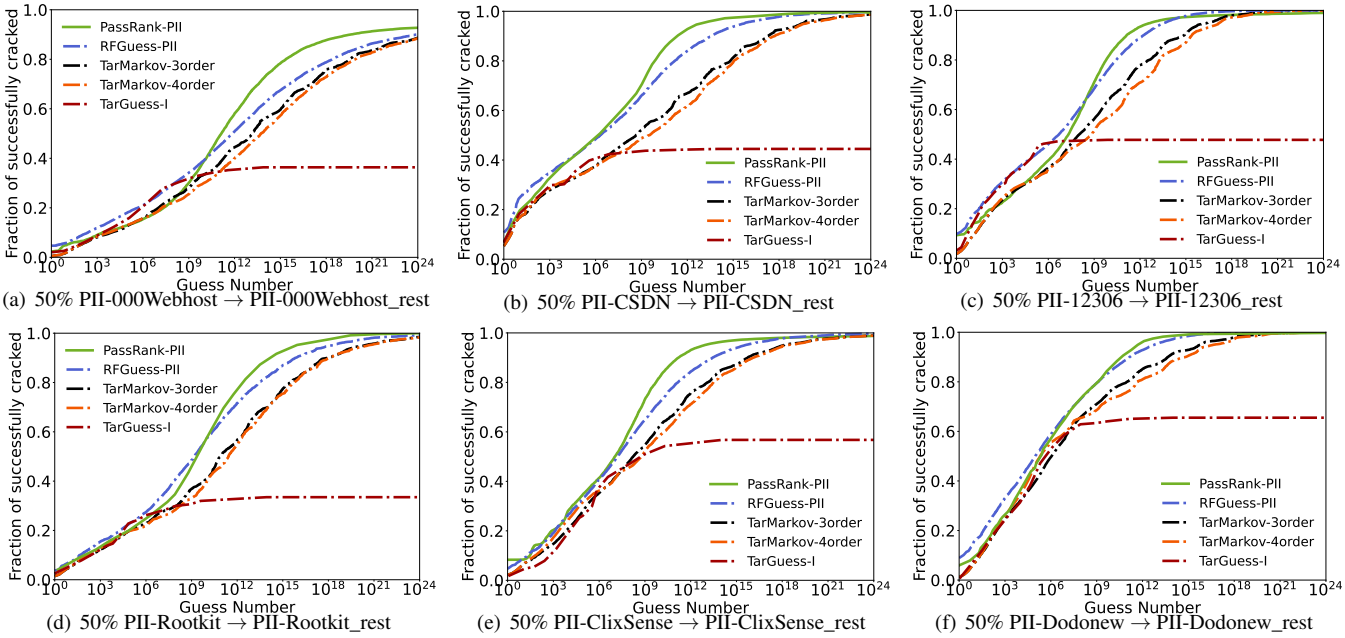
$$P(\mathrm{PW} \mid \mathrm{PII}) = \prod_{\substack{\mathrm{PW}_i \in \mathrm{PWs} \\ p_i \in \mathrm{PII}}} P(\mathrm{PW}_i \mid p_i, \mathcal{G}_\theta), \tag{11}$$

where PW denotes the password given personally identifiable information with guesser $\mathcal{G}_\theta$ and p_i are some attributes in PII.

4.1 PII matching algorithm

For passwords which contain PII [87], such as `wang1106` (i.e., $\mathbf{N}_3$: `wang` in name `leiwang`, $\mathbf{B}_2$: `1106` in birthday `19851106`, etc.), we have noticed that such passwords can be split into PII segments, such as $\mathbf{N}_3\mathbf{B}_2$. Li *et al.* [48] introduced an approach for PII matching akin to the PCFG method [39], where, for instance, $\mathbf{N}_4$ denotes a name with a length of four, such as `wang`. It's evident that when PII is already known, splitting the password into segments based on letters as the basic tokens may lead to a loss of entropy of PII [89]. Using PII segments as the basis for dividing the password can better preserve semantics as proofed in [89]. For the password `wang1106`, compared to splitting it into token levels, tagging [7, 50, 53] PII can help avoid the loss of both the name and birthday information. Therefore, we need an effective PII matching algorithm for deep generative models.

Inspired by Wang *et al.*'s work [87, 89], the first procedure in our PII matching involves designing a set of tagging rules and establishing a series of tokenized labels to represent personal information. The summarization of these labels is provided in Table 9. Our tagging rules start from 1000 as $\mathbf{N}$, where 1000 represents the complete PII, 1003 denotes the uppercase surname $\mathbf{N}_3$, 1004 represents the lowercase surname $\mathbf{N}_4$, 2000 indicates the complete birthday as $\mathbf{B}$, and so on. Based on these rules, a password containing personal information can be represented using a series of these labels. The second procedure involves enumerating all possible outcomes for each password by substituting PII labels. For instance, consider the password `wang1106`. It can be represented in ten different ways: 1004,2002; 1004,2003,2004; 1004,1,1,0,6; 1004,2003,0,6; 1004,1,1,1004; w,a,n,g,2002;



(a) 50% PII-000Webhost → PII-000Webhost_rest (b) 50% PII-CSDN → PII-CSDN_rest (c) 50% PII-12306 → PII-12306_rest

(d) 50% PII-Rootkit → PII-Rootkit_rest (e) 50% PII-ClixSense → PII-ClixSense_rest (f) 50% PII-Dodonew → PII-Dodonew_rest

Figure 6: Guessing performance of our PassRank-PII in comparison with other targeted approaches using PII (i.e., Targuess-I [87], 3/4-order TarMarkov [85], RFGuess-PII [89]) in the intra-site target guessing scenarios. Dataset_rest refers to the original dataset without including the PII training set. Our PassRank-PII demonstrates superior performance compared to our counterparts.

w,a,n,g,2003,2004; w,a,n,g,1,1,2004; w,a,n,g,2003,0,6; and w,a,n,g,1,1,0,6. In contrast to Wang *et al.*'s works [87, 89], where a single representation is selected as the definitive one for each password, our approach involves inputting all potential representations into the model during the training process. Our approach deviates from conventional machine learning paradigms, as deep learning necessitates substantial training datasets and places less emphasis on intricate feature engineering and preprocessing techniques. In this context, multiple representations can serve as a part of data augmentation [78, 93]. This means that instead of using just one representation of the data (i.e., passwords), we utilize various possible representations of the same data. Our approach can help enhance the robustness and generalization capabilities of models. We summarize our PII matching in Algorithm 3.

4.2 Experimental setup and results

Datasets. We evaluate the existing guessing approaches and our PassRank-PII model based on 6 large real-world datasets (see Table 4). As indicated in Table 3, there are three datasets originally containing diverse forms of PII. To facilitate comprehensive targeted guessing evaluations, we align the passwords that do not initially contain PII with these three PII-containing datasets through email matching. This results in a total of six datasets associated with PII, as detailed in Table 4.

Approaches for comparison. The prevailing targeted guessing models that utilize personally identifiable information (PII) predominantly consist of TarGuess-I [87], which is rooted in PCFG [90], and Targeted-Markov [85], which is built upon the Markov model [54]. RFGuess-PII [89], on the other hand, is designed using a random forest classification approach based on n-grams. It's worth noting that the original Targeted-Markov model, as proposed by Wang *et al.* [85], primarily focuses on leveraging name information. However, it can be readily extended to include additional user details such as user name, birthday, email, and so forth.

Experimental results. The current mainstream targeted guessing models employing PII mainly include the TarGuess-I [87] based on PCFG [90] and the Targeted-Markov [85] based on the Markov [54]. Note that the original Targeted-Markov proposed by Wang *et al.* [85] exploits only name information, but it can be easily extended to incorporate user name, birthday, email, etc. To accurately depict the success rates of various attack methods, we present specific result values at certain guess numbers (i.e., 10^5 and 10^8, as shown in Table 12 in Appendix C). We first conducted intra-site guessing experiments (see Figure 6) and cross-site guessing experiments (see Table 12 in Appendix C). Traditional statistical methods, i.e., Targuess-I, and traditional machine learning methods perform well with small guess numbers (i.e., $<10^3$). However, PassRank, which is based on deep reinforcement learning, outperforms them. It leads by 3.02% to 8.23% (on average 6.17%) at 10^{12} guesses. To provide a more precise representation of the experimental results, we have summarized the results of intersite guessing in Table 5. The results are similar to those of intra-site guessing, but the success rate is relatively lower due to different language usage habits.

We also perform a like-for-like performance comparison of

Table 5: Comparison of four PII-based models.[†]

Experimental setup		PassRank-PII	RFGuess-PII [89]	TarMarkov 3-order [85]	Tar-Guess-I [87]
Guessing scenario	Guess #				
50% PII-12306 ↓ 50% PII-CSDN	10	15.91%	**16.06%**	15.67%	8.41%
	10^2	**21.37%**	20.40%	20.09%	18.47%
	10^5	28.27%	27.20%	26.35%	**29.01%**
	10^8	**44.90%**	41.00%	43.27%	39.51%
	10^{15}	**70.45%**	66.58%	65.94%	56.05%
50% PII-CSDN ↓ 50% PII-ClixSense	10	4.15%	**5.20%**	4.12%	4.93%
	10^2	6.81%	**7.90%**	7.28%	5.96%
	10^5	**12.40%**	10.96%	11.45%	9.97%
	10^8	**27.83%**	25.46%	23.23%	24.85%
	10^{15}	**94.69%**	90.74%	85.17%	64.51%
50% PII-Dodonew ↓ 50% PII-000Webhost	10	2.51%	**3.07%**	2.40%	2.52%
	10^2	7.33%	**8.00%**	7.10%	6.18%
	10^5	**19.05%**	17.29%	16.50%	14.39%
	10^8	**24.84%**	22.62%	23.45%	21.72%
	10^{15}	**72.22%**	69.61%	68.86%	53.80%
50% PII-ClixSense ↓ 50% PII-12306	10	**14.82%**	11.87%	10.90%	9.12%
	10^2	**20.05%**	18.05%	17.70%	16.67%
	10^5	**32.36%**	30.06%	28.70%	29.15%
	10^8	**57.11%**	54.01%	51.48%	55.75%
	10^{15}	**99.29%**	97.33%	92.38%	78.60%
50% PII-Rootkit ↓ 50% PII-Dodonew	10	**6.96%**	5.95%	6.32%	5.47%
	10^2	**11.40%**	10.07%	10.46%	9.21%
	10^5	**16.17%**	14.88%	15.12%	12.25%
	10^7	**39.45%**	38.81%	27.73%	36.77%
	10^{14}	**89.81%**	86.10%	78.24%	56.91%
50% PII-000Webhost ↓ 50% PII-Rootkit	10	**10.82%**	8.75%	7.90%	6.76%
	10^2	**15.19%**	13.89%	14.10%	12.64%
	10^5	24.56%	**25.17%**	22.32%	21.71%
	10^8	**37.57%**	32.26%	30.34%	31.73%
	10^{15}	**93.93%**	90.45%	85.43%	60.64%

[†]A value in **bold** (attack success rate) indicates that it is the best within each respective row.

these approaches with regard to dataset size (Exp. #6–#8 in Table 12) and language. We use a mixed representation of all training datasets and then split them into partitions (e.g., 90%, 70%, 50%, etc.). In the case of extensive training datasets, it is evident that PassRank-PII is more stable and better suited for targeted password guessing using PII than RFGuess-PII [89], TarMarkov-3/4order [87], and Targuess-I [87].

5 Conditional password guessing

Attackers $\mathcal{A}$ have obtained an incomplete password of the victim by exploiting various side-channel attacks [68, 96] (shoulder surfing [22], keystroke audio feedback [24, 45], etc.), then initiates a password guessing attack. Conditional guessing models crack complete passwords given a partial password (e.g., `e***a*rt*`,`p***ord**3`). A simple way [68] to tackle this issue is by using trawling models to create numerous potential passwords. However, the task of efficiently removing irrelevant passwords from this pool can be resource-intensive and demand significant storage [96]. In this work, the symbol `*` serves as a wildcard character that acts as a stand-in for an unspecified character. For example, the pivot `e***a*rt*` signifies a category of passwords beginning with `elizabeth` and followed by five unspecified characters.

5.1 Conditional password generation

Conditional password guessing [68, 96] predicts the probability of the real passwords (PWs) conditioned on a pivot:

$$P(\text{PW} \mid pivot) = \prod_{\substack{\text{PW}_i \in \text{PWs} \\ p_i \in pivot}} P(\text{PW}_i \mid p_i, pivot, \mathcal{G}_\theta). \quad (12)$$

Finding an optimal guessing strategy that minimizes the expected guess rank is NP-hard. However, attackers, $\mathcal{A}$ could still find approximate solutions that minimize the expected guess rank. We design a BFS with pruning in Appendix A.4 and Algorithm 4 for conditional attacks. It generates a set of guesses $\mathcal{P}$, given a password pivot, the number of guesses n, and the number of masked positions m. It employs an iterative process to systematically investigate potential characters for each position. This method involves choosing the most probable deep candidates and constructing guesses.

The algorithm is not always optimal but still offers a fairly accurate approximation of the success of the optimal attacker. Whether an efficient algorithm with tighter approximation bounds exists remains a challenging open question.

5.2 Experimental setups and results

Datasets. We evaluate the existing password guessing approaches and our PassRank-Mask model based on eight large real-world password datasets (see Table 3). Five of our password datasets are from English sites and three are from Chinese sites. In strict adherence to the evaluation approach meticulously outlined in previous cutting-edge research [68, 96], we systematically employ randomly masked characters with a precisely calibrated 50% within a password. Furthermore, as a stringent criterion, we meticulously retain only those ingeniously generated pivots that contain a minimum of five masked symbols and four discernible.

Our evaluation aligns with both our model and existing methods like CWAE [68] and PassBERT [96]. We generated a substantial pool of 10^7 candidate passwords for each evaluation pivot. We classified these evaluation pivots into four categories based on the number of passwords in each dataset satisfying each pivot, denoted as N_{pivots}: (1) ***common*** if $N_{\text{pivots}} \in [1000, 1500]$; (2) ***uncommon*** if $N_{\text{pivots}} \in [50, 150]$; (3) ***rare*** if $N_{\text{pivots}} \in [10, 15]$; (4) ***super-rare*** if $N_{\text{pivots}} \in [1, 5]$. These categories provide a nuanced assessment of pivot guessing performance across different pivot frequencies. Specific illustrative examples of evaluation pivots and the corresponding passwords generated and crack rate through the utilization of PassRank-Mask via Algorithm 4, can be found in Table 6.

Experimental settings. We compare PassRank-Mask with two leading models, i.e., CWAE [68] and PassBERT [96]. The setups of PassRank-Mask are identical to those outlined in Section 3, while the other approaches are as follows:

CWAE. The CWAE we employed is in accordance with Pasquini *et al.* [68] and is implemented using the open-source[2] PasswordAE model, utilizing the default parameters.

PassBERT. For PassBERT [96] model, we use the source code[3] of Xu *et al.* [96], and follow its recommended parameters in our experiments. More specifically, to fine-tune on the pre-trained model released by PassBERT [96] (i.e., pre-training on randomly 60 million passwords sample from `Rockyou-2021`, which is a breach dataset happened in 2021),

[2] https://github.com/pasquini-dario/PLR

[3] https://github.com/snow0011/PassBertStrengthMeter

Table 6: Provided are pivot samples and the coresponding top-5 passwords cracked by PassRank-Mask in Rockyou. The main results are classified according to pivot frequency. For instance, using a pivot like "**e**aet***" in 10^7 guessing attempts yield a success rate of 95.20, in which "**elizabeth**", a well-known female first name, emerges as the most probable (top-1) password.

Pivot	***Common***			***Uncommon***			***Rare***			***Super-rare***		
Rank	****nnie*****	***ar*an*****	***u****123**	**l*ng***n***	***i**e*bo***	**e***a*et***	**1*3**67**0**	**my**ss**r***	**sli*****w**	***a*r*l*pp*r**	****o*eso*c*r**	**1*51*1*86****
1	minniewow	darkangel	august123	longhuong	fisherboy	elizabeth	1234567890	mypassword	sliveshow	hairclipper	ilovesoccor	10519148650
2	hunniebee	farhana11	bunnie123	longhorn1	singerboy	ericareta	1234567880	mypassw0rd	sliv0show		ilovesoccer	13512108611
3	hannie123	harkangel	hungry123	longhorns	silverboy	elixabeth	1234567650	mylesstorm	slideshow		llovesoccer	13510138639
4	jonniece1	maryangel	butter123	langhorn!	mickeyboy	elysabeth	1334567980	mypasswor1	slidwshow			
5	minnie123	marian123	turtle123	longhorn7	cinderbob	**elezabeth**	1234567000	mypasswor8	slinkylaw			
Crack (%)	**99.13**	**78.26**	**86.91**	**69.48**	**83.72**	**95.20**	**72.67**	**91.19**	**85.09**	**28.53**	**43.71**	**56.76**

Table 7: Comparison of three conditional guessing models.[†]

Experimental setup		PassRank-Mask	Pass-BERT [96]	CWAE [68]
Guessing scenario	Guess number			
CSDN → LinkedIn	10	28.12%	25.80%	29.13%
	10^4	59.80%	56.13%	61.19%
	10^7	75.86%	71.88%	85.19%
CSDN → 000Webhost	10	18.59%	16.82%	15.44%
	10^4	51.79%	49.86%	44.56%
	10^7	54.01%	53.61%	50.58%
Wishbone → Dodonew	10	24.14%	23.44%	19.19%
	10^4	56.00%	55.69%	44.34%
	10^7	67.65%	64.78%	57.58%
Wishbone → Taobao	10	30.17%	25.11%	27.70%
	10^4	60.29%	56.42%	52.14%
	10^7	73.77%	67.63%	64.37%
Rockyou → LinkedIn	10	23.12%	13.38%	16.06%
	10^4	61.77%	58.16%	44.01%
	10^7	78.52%	62.35%	72.69%
Rockyou → Clixsense	10	26.91%	25.12%	24.84%
	10^4	68.01%	58.69%	55.87%
	10^7	80.13%	66.99%	70.19%

[†]A value with dark gray (resp. light gray) represents the highest one (resp. 2nd one).

we train a model consisting of two hidden layers with each 128 hidden size, gelu hidden activation and Dropout rate 0.1 (namely, the medium config model in PassBERT [96]).

Experimental results. We design six scenarios for cross-site guessing. To evaluate the model's guessing performance at various guessing levels, we set three levels of guess numbers: low, moderate, and high (i.e., 10, 10^4, 10^7). Based on the experimental results in Table 7, PassRank-Mask achieves the state-of-the-art level in all three guessing levels across the five scenarios. Notably, in the Rockyou → Clixsense scenario at 10^7 guesses, it outperforms better than CWAE [68] by 14.16%. Across other scenarios, at 10^7 guesses, PassRank-Mask shows an average success rate improvement of 7.28%, providing significant evidence of its superior cracking performance compared to PassBERT [96] and CWAE [68].

Moreover, to comprehensively assess our model's cracking ability and investigate its generalization capacity, we further design additional attack scenarios. We study the model's cracking performance when cracking passwords across languages (e.g., Chinese → English, and vice versa) to investigate its generalization. We divide the password sets into Chinese (CSDN, Dodonew, Taobao), English (Wishbone, 000Webhost, LinkedIn), mixed Chinese (i.e., mix all Chinese passwords), mixed English password sets (mix all English passwords), and all mixed. Based on these, we conduct mutual attack tests to assess their success rates. The specific combinations of attack and target password sets, as well as the experimental results, are presented in Appendix C and Table 11 of the paper. From the experimental results, it is evident that PassRank-Mask achieves the best performance in most cross-site scenarios, with an average performance improvement of 5.64%. Notably, in scenarios involving Chinese mixed, English mixed, and all mixed as the training sets, PassRank-Mask achieves the best performance in nearly all scenarios, with an average performance improvement of 7.09%. This significantly demonstrates the generalization ability and performance of the PassRank-Mask.

6 Implications and insights

6.1 Password strength meters

One key application of password cracking algorithms is to accurately measure password strength from an attacker's perspective. Password strength meters (PSMs) [12, 19, 20, 25, 27, 66, 80, 84] show the strength of passwords to help users change a weak password to a strong one, since users' perception of password security is generally error-prone [84]. Labeling passwords as weak may cause usability problems while classifying them as strong could introduce security issues. Additionally, we have implemented a PSM. To assess the effectiveness of our meter, we gauge its performance using two accuracy metrics [28, 84] to detect unsafe errors [57].

On the accuracy of PSMs. To obtain general results across meters in the real world, we refer to practices [28, 84] that count how meters' guesses for a given password are correlated with occurrence frequencies in an evaluation set using the weighted spearman coefficient (r_s) as Golla *et al.* [92]:

$$r_s = 1 - \frac{6\sum d_i^2}{n(n^2-1)} \tag{13}$$

where $d_i = \text{rank}(\texttt{PSM}) - \text{rank}(\texttt{FRE})$, represents the difference between the ranks in PSMs and real frequency, and n is the number of passwords considered in the calculation.

Each of these datasets is obtained by sampling 0.1 M. As shown in Table 8, our meter achieves better accuracy than the Zxcvbn meter, and in half of the cases, it is better than Min_auto [81]. In evaluation sets using CSDN, PassRank has a correlation of 0.728, compared with 0.643 and 0.514 in

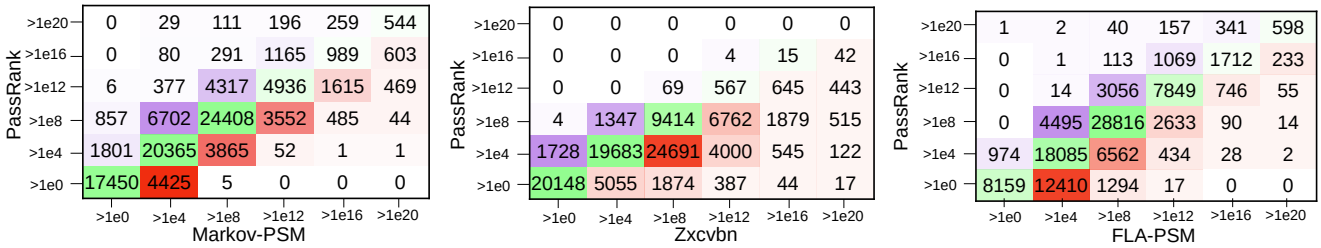


Figure 7: Unsafe errors in PSMs. The utmost significance of this lies in the initial row, as it pertains to the quantity of passwords that are grossly overestimated by other meters. Our PassRank meter assesses a total of 6,377 passwords as being guessable within the range of 1 to 10^4 guessing attempts, while the Zxcvbn [91] meter categorizes them as requiring more than 10^4 attempts. PassRank and FLA-PSM [57] show similar performance on unsafe errors. The exaggerations of strength are visually and graphically represented in various shades of bold red, while underestimations are depicted in deep shades of purple, and precise evaluations are denoted in soothing shades of green. The color intensity escalates significantly in accordance with the number of passwords within a specific category, following the setting of [28, 57, 95].

Table 8: Accuracy (r_s) across PSMs, determined through the weighted Spearman correlation of five models with evaluation sets. A **bold** value signifies the best performance among them.

Dataset	**English**		**Chinese**	
PSMs	LinkedIn	Rockyou	CSDN	Dodonew
PassRank	**0.652**	0.511	**0.728**	**0.535**
FLA-PSM [57]	0.367	0.355	0.643	0.322
Zxcvbn [91]	0.579	0.463	0.514	0.475
Markov-PSM [12]	0.201	0.337	0.268	0.297
Min_auto [81]	0.558	**0.694**	0.575	0.423

FLA-PSM [57] and Zxcvbn [91]. Among them, Markov-PSM performs the worst. Our PassRank-PSM provides a more precise assessment of password strength according to Table 8.

Unsafe errors. The unsafe errors [57] refer to rating easy-to-guess passwords as strong ones or namely, overestimation of password strength[5]. We investigate these meters' unsafe errors and show the average results using a 10w password randomly sampled from CSDN. Our PassRank approach demonstrated superior accuracy, with up to 1.7 times fewer unsafe errors compared to Markov-PSM [12] and FLA [57] as PSM, as shown in Figure 7. We used our tuned PassRank meter implementation described in Section 3. In addition, we find that, compared to others, the Zxcvbn meter's errors are often at very low guess numbers, which can be particularly unsafe. Overestimation of password strength can be disastrous since the seemingly secure passwords could be actually not safe or very vulnerable. For example, for the 10,000 most likely passwords, Zxcvbn [91] makes 183 utmostly unsafe errors.

6.2 Honeywords system

Personal honeyword system. At CCS'13, Juels and Rivest [37] introduced a honeyword mechanism for password file compromise detection. This system generates $k-1$ honeywords per account (e.g., $k = 20$ in [37]) and stores them alongside the actual password. When crafting honeywords for a user [37], the advanced method meticulously involves selecting password structures and substrings with similar frequency patterns to real passwords. The ultimate goal [6] is to meticulously create a set of honeywords that closely and convincingly mimic the characteristics of genuine passwords.

As a leading password model, PassRank-PII can generate honeywords for password leak detection. In this scenario, model size and generation speed aren't crucial. The server generates 20 to 40 honeywords per account, following [88]. If an attacker uses a honeyword, it triggers an alert, notifying the administrator of a potential password breach.

Conditional honeyword system. The earlier honeywords system was capable of resisting trawling guessing and targeted attacks [6, 88] but was defenseless against conditional attacks. Conditional password guessing (this work and [68, 96]) is a crucial scenario where, for a user's password, we can employ a conditional attack to obscure a certain percentage (e.g., 40 $\sim$ 60% [68]) of randomly selected characters. Subsequently, using the conditional attack models, passwords with probabilities and user password probabilities equal to those of the masked characters can be generated. This can, to some extent, mitigate the security risks posed by such attacks, which were not considered in previous research.

7 Conclusion

This paper, *for the first time*, introduces reinforcement learning for password guessing and designs three new guessing models for the three most representative guessing scenarios: trawling guessing, targeted guessing based on PII, and conditional password guessing. Extensive experiments with 12 real-world datasets demonstrate the effectiveness and scalability of our models. Further, we propose a password strength meter to mitigate the risks of real-world guessing attacks. This work provides a brand new technical route for modeling users' password guessability and opens up new directions for designing effective password guessing models.

[5] We have obtained the approval from CMU for our research and usage of the password guessability service (PGS), which can be accessed to get min_auto guess number of passwords via https://pgs.ece.cmu.edu/.

References

[1] *Yahoo's 2013 Email Hack Actually Compromised Three Billion Accounts*, 2013. https://www.wired.com/story/yahoo-breach-three-billion-accounts/.

[2] *Massive data leak exposes 700 million LinkedIn users' information*, Jun. 2021. https://www.bleepingcomputer.com/news/security/google-microsoft-can-get-your-passwords-via-web-browsers-spellcheck/.

[3] *Google, Microsoft can get your passwords via web browser's spellcheck*, Sep. 2022. https://www.bleepingcomputer.com/news/security/google-microsoft-can-get-your-passwords-via-web-browsers-spellcheck/.

[4] *Hacker claims to be selling Twitter data of 400 million users*, Dec. 2022. https://www.bleepingcomputer.com/news/security/hacker-claims-to-be-selling-twitter-data-of-400-million-users/.

[5] *DIGITAL AROUND THE WORLD*, 2023. https://datareportal.com/global-digital-overview.

[6] Akshima, D. Chang, A. Goel, S. Mishra, and S. K. Sanadhya. Generation of secure and reliable honeywords, preventing false detection. *IEEE Trans. Dependable Secur. Comput.*, 16(5):757–769, 2019.

[7] A. Amini, T. Liu, and R. Cotterell. Hexatagging: Projective dependency parsing as tagging. In *Proc. ACL 2023*, pages 1453–1464.

[8] C. M. Bishop and N. M. Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.

[9] J. Bonneau. *Guessing human-chosen secrets*. PhD thesis, University of Cambridge, UK, 2012.

[10] J. Bonneau, C. Herley, P. van Oorschot, and F. Stajano. Passwords and the evolution of imperfect authentication. *Commun. ACM*, pages 78–87, 2015.

[11] J. Bonneau, C. Herley, P. C. Van Oorschot, and F. Stajano. The request to replace passwords: A framework for comparative evaluation of web authentication schemes. In *Proc. IEEE S&P 2012*, pages 553–567.

[12] C. Castelluccia, M. Dürmuth, and D. Perito. Adaptive password-strength meters from Markov models. In *Proc. NDSS 2012*.

[13] H. S. Chang, M. C. Fu, J. Hu, and S. I. Marcus. An adaptive sampling algorithm for solving markov decision processes. *Operations Research*, 53:126–139, 2005.

[14] G. Chaslot, S. Bakkes, I. Szita, and P. Spronck. Monte-carlo tree search: A new framework for game ai. In *Proc. AAAI 2008*, pages 216–217.

[15] C. Chen, S. Mu, W. Xiao, Z. Ye, L. Wu, and Q. Ju. Improving image captioning with conditional generative adversarial nets. In *Proc. AAAI 2019*, pages 8142–8150.

[16] J. Chevelu, T. Lavergne, Y. Lepage, and T. Moudenc. Introduction of a new paraphrase generation tool based on monte-carlo sampling. In *Proc. ACL 2009*, pages 249–252.

[17] K. Cho, B. van Merrienboer, C. Gulcehre, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Proc. EMNLP 2014*.

[18] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *Proc. NIPS 2017*, pages 1–9.

[19] X. de Carné de Carnavalet and M. Mannan. From very weak to very strong: Analyzing password-strength meters. In *Proc. NDSS 2014*.

[20] X. de Carné de Carnavalet and M. Mannan. A large-scale evaluation of high-impact password strength meters. *ACM Trans. Inf. Syst. Secur.*, 18(1):1:1–1:32, 2015.

[21] M. Dell'Amico and M. Filippone. Monte carlo strength evaluation: Fast and reliable password checking. In *Proc. ACM CCS 2015*.

[22] G. Dessouky, T. Frassetto, and A. Sadeghi. Hybcache: Hybrid side-channel-resilient caches for trusted execution environments. In *Proc. USENIX SEC 2020*, pages 451–468.

[23] J. Devlin, M. Chang, K. Lee, and K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proc.NAACL-HLT 2019*, pages 4171–4186.

[24] X. Dong, Z. Shen, J. Criswell, A. L. Cox, and S. Dwarkadas. Shielding software from privileged side-channel attacks. In *Proc. USENIX SEC 2018*, pages 1441–1458.

[25] S. Egelman, A. Sotirakopoulos, K. Beznosov, and C. Herley. Does my password go up to eleven?: The impact of password meters on password selection. In *Proc. ACM CHI 2013*, pages 2379–2388.

[26] L. Gao, J. Schulman, and J. Hilton. Scaling laws for reward model overoptimization. In *Proc. ICML 2023*, pages 10835–10866, 2023.

[27] M. Golla and M. Dürmuth. On the accuracy of password strength meters. In *Proc. ACM CCS 2018*, pages 1567–1582.

[28] M. Golla, M. Wei, J. Hainline, L. Filipe, M. Dürmuth, E. Redmiles, and B. Ur. "what was that site doing with my facebook password?" designing password-reuse notifications. In *Proc. ACM CCS 2018*.

[29] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[30] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.

[31] D. B. Grimes, D. R. Rashid, and R. P. N. Rao. Learning nonparametric models for probabilistic imitation. In *Proc. NIPS 2006*, pages 521–528.

[32] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proc.CVPR2016*, pages 770–778.

[33] B. Hitaj, P. Gasti, G. Ateniese, and F. Perez-Cruz. Passgan: A deep learning approach for password guessing. In *Proc. ACNS 2019*.

[34] J. Ho and S. Ermon. Generative adversarial imitation learning. In *Proc. NIPS 2016*, pages 4565–4573.

[35] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[36] S. Houshmand, S. Aggarwal, and R. Flood. Next gen PCFG password cracking. *IEEE Trans. Inf. Forensics Secur.*, 10(8):1776–1791, 2015.

[37] A. Juels and R. L. Rivest. Honeywords: Making password-cracking detectable. In *Proc. ACM CCS 2013*, pages 145–160.

[38] J. Justesen and T. Høholdt. Maxentropic markov chains. *IEEE Trans. Inf. Theory*, pages 665–667, 1984.

[39] J. Justesen and K. J. Larsen. On probabilistic context-free grammars that achieve capacity. *Inf. Control.*, pages 268–285, 1975.

[40] M. J. Keith, B. B. M. Shao, and P. J. Steinbart. The usability of passphrases for authentication: An empirical field study. *Int. J. Hum. Comput. Stud.*, pages 17–28, 2007.

[41] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proc. ICLR 2015*, pages 1–15.

[42] D. P. Kingma and M. Welling. Auto-encoding variational bayes. In *Proc. ICLR 2014*, pages 1–14.

[43] D. Kumor, J. Zhang, and E. Bareinboim. Sequential causal imitation learning with unobserved confounders. In M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P. Liang, and J. W. Vaughan, editors, *Proc. NIPS 2021*, pages 14669–14680.

[44] M. Kwon, S. M. Xie, K. Bullard, and D. Sadigh. Reward design with language models. In *Proc. NIPS 2023*.

[45] A. Kwong, W. Wang, J. Kim, J. Berger, D. Genkin, E. Ronen, H. Shacham, R. S. Wahby, and Y. Yarom. Checking passwords on leaky computers: A side channel analysis of chrome's password leak detect protocol. In *Proc. USENIX SEC 2023*, pages 7107–7124.

[46] S. Levine and P. Abbeel. Learning neural network policies with guided policy search under unknown dynamics. In *Proc. NIPS 2014*.

[47] S. Li, Z. Wang, R. Zhang, C. Wu, and H. Luo. Mangling rules generation with density-based clustering for password guessing. *IEEE Trans. Dependable Secur. Comput.*, pages 3588–3600, 2023.

[48] Y. Li, H. Wang, and K. Sun. A study of personal information in human-chosen passwords and its security implications. In *Proc. IEEE INFOCOM 2016*, pages 1–9.

[49] K. Lin, D. Li, X. He, M. Sun, and Z. Zhang. Adversarial ranking for language generation. In *Proc. NIPS 2017*, pages 3155–3165.

[50] M. Lindemann, A. Koller, and I. Titov. Compositional generalization without trees using multiset tagging and latent permutations. In *Proc. ACL 2023*, pages 14488–14506.

[51] P. Liu, J. Blocki, and W. Bai. Confident monte carlo: Rigorous analysis of guessing curves for probabilistic password models. In *Proc. IEEE S&P 2023*, pages 626–644.

[52] X. Liu, K. Ji, Y. Fu, W. Tam, Z. Du, Z. Yang, and J. Tang. P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In *Proc. ACL 2022*, pages 61–68.

[53] L. Loukas, M. Fergadiotis, I. Chalkidis, E. Spyropoulou, P. Malakasiotis, I. Androutsopoulos, and G. Paliouras. Finer: Financial numeric entity recognition for XBRL tagging. In *Proc. ACL 2022*, pages 4419–4431.

[54] J. Ma, W. Yang, M. Luo, and N. Li. A study of probabilistic password models. In *Proc. IEEE S&P 2014*, pages 689–704.

[55] J. MacQueen et al. Some methods for classification and analysis of multivariate observations. In *Proc. Berkeley symposium on mathematical statistics and probability 1967*, volume 1, pages 281–297.

[56] P. Mahmoudieh, D. Pathak, and T. Darrell. Zero-shot reward specification via grounded natural language. In *Proc. ICML 2022*, pages 14743–14752.

[57] W. Melicher, B. Ur, S. Komanduri, L. Bauer, N. Christin, and L. F. Cranor. Fast, lean and accurate: Modeling password guessability using neural networks. In *Proc. USENIX SEC 2016*, pages 175–191.

[58] N. Metropolis and S. Ulam. The monte carlo method. *Journal of the American statistical association*, 44:335–341, 1949.

[59] K. P. Murphy. *Machine learning: a probabilistic perspective*. MIT press, 2012.

[60] A. Narayanan and V. Shmatikov. Fast dictionary attacks on passwords using time-space tradeoff. In *Proc. ACM CCS 2005*, pages 364–372.

[61] M. Orsini, A. Raichuk, L. Hussenot, D. Vincent, R. Dadashi, S. Girgin, M. Geist, O. Bachem, O. Pietquin, and M. Andrychowicz. What matters for adversarial imitation learning? In *Proc. NIPS 2021*, pages 14656–14668.

[62] M. Osborne, A. Lall, and B. V. Durme. Exponential reservoir sampling for streaming language models. In *Proc.ACL 2014*, pages 687–692.

[63] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. In *Proc. NIPS 2022*, pages 27730–27744.

[64] L. Page, S. Brin, R. Motwani, and T. Winograd. The pagerank citation ranking: Bringing order to the web. 1999.

[65] B. Pal, T. Daniel, R. Chatterjee, and T. Ristenpart. Beyond credential stuffing: Password similarity models using neural networks. In *Proc. IEEE S&P 2019*, pages 417–434.

[66] D. Pasquini, G. Ateniese, and M. Bernaschi. Interpretable probabilistic password strength meters via deep learning. In *Proc. ESORICS 2020*.

[67] D. Pasquini, M. Cianfriglia, G. Ateniese, and M. Bernaschi. Reducing bias in modeling real-world password strength via deep learning and dynamic dictionaries. In *Proc. USENIX SEC 2021*, pages 821–838.

[68] D. Pasquini, A. Gangwal, G. Ateniese, M. Bernaschi, and M. Conti. Improving password guessing via representation learning. In *Proc. IEEE S&P 2021*, pages 265–282.

[69] J. Peters, K. Mulling, and Y. Altun. Relative entropy policy search. In *Proc. AAAI 2010*, volume 24, pages 1607–1612.

[70] M. Prerad. *20 Biggest Data Breaches of 2013*, 2013. https://www.linkedin.com/today/post/article/20140224081155-67886711-20-biggest-data-breaches-of-2013.

[71] M. L. Puterman. Markov decision processes. volume 2 of *Handbooks in Operations Research and Management Science*, pages 331–434. 1990.

[72] S. Sahin and F. Li. Don't forget the stuffing! revisiting the security impact of typo-tolerant password authentication. In *Proc. ACM CCS 2021*, pages 252–270.

[73] Y. Schroecker and C. L. I. Jr. State aware imitation learning. In *Proc. NIPS 2017*, pages 2911–2920.

[74] C. E. Shannon. A mathematical theory of communication. *Bell Syst. Tech. J.*, pages 379–423, 1948.

[75] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano. Learning to summarize with human feedback. pages 3008–3021.

[76] W. Sun, A. Venkatraman, G. J. Gordon, B. Boots, and J. A. Bagnell. Deeply aggrevated: Differentiable imitation learning for sequential prediction. In *Proc. ICML 2017*, pages 1–17.

[77] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[78] M. A. Tanner and W. H. Wong. The calculation of posterior distributions by data augmentation. *Journal of the American statistical Association*, 82(398):528–540, 1987.

[79] I. Tolstikhin, O. Bousquet, S. Gelly, and B. Schoelkopf. Wasserstein auto-encoders. In *Proc. ICLR 2018*, pages 1–16.

[80] B. Ur, P. G. Kelley, S. Komanduri, and et al. How does your password measure up? The effect of strength meters on password creation. In *Proc. USENIX SEC 2012*, pages 65–80.

[81] B. Ur, S. M. Segreti, L. Bauer, N. Christin, L. F. Cranor, S. Komanduri, D. Kurilova, M. L. Mazurek, W. Melicher, and R. Shay. Measuring real-world accuracies and biases in modeling password guessability. In *Proc. USENIX SEC 2015*, pages 463–481.

[82] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, . Kaiser, and I. Polosukhin. Attention is all you need. In *Proc. NeurIPS 2017*, pages 5998–6008.

[83] R. Veras, C. Collins, and J. Thorpe. On the semantic patterns of passwords and their security impact. In *Proc. NDSS 2014*, pages 1–16.

[84] D. Wang, X. Shan, Q. Dong, Y. Shen, and C. Jia. No single silver bullet: Measuring the accuracy of password strength meters. In *Proc. USENIX SEC 2023*, pages 947–964.

[85] D. Wang and P. Wang. The emperor's new password creation policies. In *Proc. ESORICS 2015*, pages 456–477.

[86] D. Wang, P. Wang, D. He, and Y. Tian. Birthday, name and bifacial-security: Understanding passwords of chinese web users. In *Proc. USENIX Sec 2019*, pages 1537–1555.

[87] D. Wang, Z. Zhang, P. Wang, J. Yan, and X. Huang. Targeted online password guessing: An underestimated threat. In *Proc. ACM CCS 2016*, pages 1242–1254.

[88] D. Wang, Y. Zou, Q. Dong, Y. Song, and X. Huang. How to attack and generate honeywords. In *Proc. IEEE S&P 2022*, pages 966–983.

[89] D. Wang, Y. Zou, Z. Zhang, and K. Xiu. Password guessing using random forest. In *Proc. USENIX SEC 2023*, pages 965–982.

[90] M. Weir, S. Aggarwal, B. de Medeiros, and B. Glodek. Password cracking using probabilistic context-free grammars. In *Proc. IEEE S&P 2009*, pages 391–405.

[91] D. Wheeler. zxcvbn: Low-budget password strength estimation. In *Proc. USENIX SEC 2016*, pages 157–173.

[92] C. Wissler. The spearman correlation formula. *Science*, 22(558):309–311, 1905.

[93] Q. Xie, Z. Dai, E. Hovy, T. Luong, and Q. Le. Unsupervised data augmentation for consistency training. *Proc. NIPS 2020*.

[94] Y. Xiong. *Nearly one billion people in China had their personal data leaked*, Jul. 2022. https://edition.cnn.com/2022/07/05/china/china-billion-people-data-leak-intl-hnk/.

[95] M. Xu, C. Wang, J. Yu, J. Zhang, K. Zhang, and W. Han. Chunk-level password guessing: Towards modeling refined password composition representations. In *Proc. ACM CCS, 2021*, pages 5–20.

[96] M. Xu, J. Yu, X. Zhang, C. Wang, S. Zhang, H. Wu, and W. Han. Improving real-world password guessing attacks via bi-directional transformers. In *Proc. USENIX SEC 2023*, pages 1001–1018.

[97] L. Yu, W. Zhang, J. Wang, and Y. Yu. Seqgan: Sequence generative adversarial nets with policy gradient. In *Proc. AAAI 2017*.

[98] V. Zimmermann. *From the quest to replace passwords towards supporting secure and usable password creation*. PhD thesis, Technical University of Darmstadt, Germany, 2021.

A Password algorithm details

This section provides supplementary implementation details for the trawling password guessing part in Section 3, the targeted password guessing using PII part in Section 4, and the conditional password guessing part in Section 5.

A.1 Adversarial ranking guessing framework

The ranker $\mathcal{R}_\varphi$ continually assesses and scores the performance of the guesser $\mathcal{G}_\theta$ while being guided by the expert $\mathcal{E}$ to improve the generative capabilities of the guesser. Consequently, this process can be viewed as an ongoing interaction where $\mathcal{G}_\theta$ and $\mathcal{R}_\varphi$ participate in a minimax game. We have summarized the algorithm for adversarial training of the ranker and guesser as follows:

Algorithm 1: Adversarial ranking framework.

Input: Password set $\mathcal{D}$, training epoch $\mathcal{T}$, iteration count $\mathcal{J}$, $\mathcal{K}$, state space $\mathbf{S}$, action space $\mathbf{A}$, differentiable policy $\pi_\theta(a \mid s)$, learning rate α, β.
Output: Policy $\pi_\theta(a \mid s)$.

Randomly initialize parameters θ and φ;
for $t \leftarrow 0$ **to** $\mathcal{T}-1$ **do**
  /* Training ranker $\mathcal{R}_\varphi$*/
  for $k \leftarrow 0$ **to** $\mathcal{J}-1$ **do**
    Generate samples $\mathcal{S}^- \sim \mathcal{G}_\theta$ and $\mathcal{S}^+ \sim \mathcal{P}_h$ in $\mathcal{D}$;
    Mixup $\mathcal{S}_i^M = \lambda_i \mathcal{S}^+ + (1-\lambda_i)\mathcal{S}^-$, where $\lambda_i \sim U(N)$;
    Get the rank distribution $\mathcal{T}_{\theta,\varphi}(\mathcal{S}_i^M, \mathcal{E})$ via Equation. (7);
    $\varphi \leftarrow \varphi - \alpha \sum_{i=0}^{T-1} \nabla_\varphi \mathcal{T}_{\theta,\varphi}(\mathcal{S}_i^M, \mathcal{E}) \log \lambda_i$;
  /* Training guesser $\mathcal{G}_\theta$*/
  Generate a trajectory $\tau = \{s_1, a_1, s_2, a_2, \ldots, s_T, a_T\}$ based on the strategy $\pi_\theta(a \mid s)$;
  for $t \leftarrow 0$ **to** $\mathcal{K}-1$ **do**
    $\delta \leftarrow \mathcal{T}_{\theta,\varphi}(\mathrm{PW}_{1:t-1}, \mathcal{E})$via MC search;
    $\theta \leftarrow \theta + \beta \mathop{\mathbb{E}}_{\mathrm{PW}_{1:T} \sim \mathcal{G}_\theta} \left[\sum_{t=1}^{T} \sum_{w_t \in \mathcal{V}} \nabla_\theta \delta_t \log \pi_\theta(w_t | \mathrm{PW}_{1:t-1}) \right]$;
return $\pi_\theta(a \mid s)$

A.2 Password guessing algorithm

In order to conform with the actual password length distribution, we have devised an adaptive password generation algorithm. At the outset, we initialize the process with an empty stack $\mathcal{Q}$ for storing prefixes and a lookup table L for probability storage as ASCII 32-126 we used in Table 10.

The algorithm proceeds through iterations, where it retrieves a prefix from the stack and employs our model to predict character probabilities. For each character within the charset $\mathcal{V}$, it computes a new probability. If this probability surpasses the defined threshold, or if the character is E_s, the prefix is deemed a valid password and is included in the set $\mathcal{P}$. However, if the probability falls short or the character is not E_s, the prefix is extended by appending the character, its probability is updated in L, and it is subsequently returned to the stack for further processing. This recursive procedure continues until the stack is empty. Ultimately, the algorithm furnishes the set of generated passwords, denoted as $\mathcal{P}$.

Algorithm 2: Password guessing algorithm.

Input: The probability threshold $\mathcal{T}$ of generated passwords.
Output: Passwords set $\mathcal{P}$.

Init LUT: look up table{key : $prefix$, value : $prob$}
$\mathcal{Q}.push(\mathrm{B}_s.tokenize())$ /* Implement a tree using a list, build a search tree using BFS, and sort nodes based on heuristic cost. */
while $!\mathcal{Q}.empty()$ **do**
  $cur_prefix \leftarrow \mathcal{Q}.pop()$
  $next_prob \leftarrow guesser\ \mathcal{G}_\theta(cur_prefix)$
  $char_set\ \mathcal{V} \leftarrow$ top_ω (cur_prefix, $adaptive_beam_width$ ω)
  for c *in* $char_set\ \ \mathcal{V}$ **do**
    $cur_prob \leftarrow LUT[cur_prefix] * next_prob[c]$
    if $cur_prob > \alpha$ **then**
      if $c == E_s$ **then**
        $\mathcal{P}.append(cur_prefix)$
      else
        $\mathcal{Q}.push(cur_prefix + c)$ /* The entire algorithm generates passwords by traversing each a trajectory τ that meets the criteria. */
        $LUT[cur_prefix + \mathrm{c}] \leftarrow cur_prob$
return $\mathcal{P}$

A.3 Our PII matching algorithm

Algorithm 3: Our PII matching algorithm.

Input: Passwords set $\mathcal{X} = \{pw_1, pw_2, \ldots, pw_n\}$, mapping from PII to labels $pattern2tags$. /* Pattern2tags are a series of pre-defined (personal information fragment, tag) tuples, indicating the tagging to be applied to each PII. */
Output: Passwords tagging PII structures $\mathcal{P}$.

for $pattern, tag$ **in** $pattern2tags$ **do**
  $pos = \mathcal{P}.match(pattern)$;/* Perform string matching to ascertain the presence of matchable PII. */
  if $pos! = -1$ **then**
    for $i = 0$ to $pattern.length()$ **do**
      for $index = i + pos$ **do**
        $tag_pwd = \mathcal{P}.replace(index, tag)$
      $tag_pwds = PII_match(tag_pwd, pattern2tags)$;
    /* Conducting DFS recursively, with tag_pwds storing the matched password strings. */
    for i = *0* to $pattern.length()$ **do**
      $tag_pwd = pwd.replace(pos + i, pattern[i])$;
for tag_pwd **in** tag_pwds **do**
  $\mathcal{P}.append(unique_tag(tag_pwd))$;/* Represent each PII with a single tag, eliminating unnecessary tags. */
return $\mathcal{P}$

The first step involves categorizing all possible personal information transformations and representing them with corresponding labels. The second step is to exhaustively generate all possible representations of passwords using the transformations and labels identified in the first step. The third step entails learning the required representations through the model from all the generated representations.

A.4 Pruned BFS for conditional attack

Determining an optimal guessing strategy to minimize the expected guess rank is a computationally challenging problem, classified as NP-hard. Nonetheless, adversaries can seek approximate solutions that aim to minimize the expected guess rank to a reasonable extent. While the algorithm in question may not achieve optimality, it does offer a valuable approximation of the success achievable by an optimal attacker. The inquiry into the existence of an efficient algorithm with approximation bounds remains an unanswered question.

Algorithm 4: Pruned BFS for conditional attack.

Input: Password pivot p, guess number n, masked number m.
Output: Passwords set $\mathcal{P}$.

```
Search width ω = ⌈ ⁿ√n ⌉; /* Calculate the approximate possible
  states required for each position p. */
Empty queue Q with parallelizable pop operations;
Initialize the queue Q, and Q.push((initial_state, prob = 1.0));
for masked position p_1 to p_n do
    while !Q.empty() do
        cur_tuple = Q.parallel_pop();
        for cur_pivot, cur_pivot_prob in cur_tuple do
            next_probs = guesser G_θ(cur_pivots);
            Select top-ω next_probs and chars /* Select ω
              candidate characters for each p_i in pivots. */
            for char,next_prob in chars,next_probs do
                next_pivot = cur_pivot.replace(p_i, char)
                new_prob = cur_pivot_prob*next_prob
                Q.push(next_pivot, new_prob) /* Search for
                  possible characters at each position p_i. */
Ouput password set P = Q.parallel_pop()
return P
```

B Proof of the results

For readability, we provide the notation in Table 9 and the detailed derivation of the equation used in this paper.

B.1 Proof of the policy gradient

We now proof our proposed policy gradient update algorithm (in Table 3.2). The goal of reinforcement learning [77] is to learn a policy $\pi_\theta(a \mid s)$ that maximizes the expected return $\mathfrak{L}(\theta) = \mathbb{E}_{x_{1:T}\sim P_\theta(x_{1:T})}\left[\mathcal{T}\left(x_{1:T}\right)\right]$, as mentioned at Section 1.1. One direct method is to search the policy space directly to find the optimal policy, known as policy search. Policy search [46, 69] is essentially an optimization problem and can be categorized into gradient-based optimization and non-gradient-based optimization. Compared to methods based on value functions, policy search may not require value functions and can directly optimize the policy. Parameterized policies can handle continuous states and directly learn policies.

Policy gradient [77] is a gradient-based reinforcement learning method. Assuming that $\pi_\theta(a \mid s)$ is a continuously differentiable function with respect to θ, we can optimize the parameter θ using gradient ascent to maximize the objective function $\mathfrak{L}(\theta)$, of which the derivative of the objective function w.r.t. the policy parameters θ is given by:

$$
\begin{aligned}
\frac{\partial \mathfrak{L}(\theta)}{\partial \theta} &= \frac{\partial}{\partial \theta}\int P_\theta(\tau)\mathcal{T}(\tau)\mathrm{d}\tau \\
&= \int \left(\frac{\partial}{\partial \theta}P_\theta(\tau)\right)\mathcal{T}(\tau)\mathrm{d}\tau \\
&= \int P_\theta(\tau)\left(\frac{1}{P_\theta(\tau)}\frac{\partial}{\partial \theta}P_\theta(\tau)\right)\mathcal{T}(\tau)\mathrm{d}\tau \\
&= \int P_\theta(\tau)\left(\frac{\partial}{\partial \theta}\log P_\theta(\tau)\right)\mathcal{T}(\tau)\mathrm{d}\tau \\
&= \mathbb{E}_{\tau\sim P_\theta(\tau)}\left[\frac{\partial}{\partial \theta}\log P_\theta(\tau)\mathcal{T}(\tau)\right],
\end{aligned}
\tag{14}
$$

where $\frac{\partial}{\partial \theta}\log P_\theta(\tau)$ represents the partial derivative of the function $\log P_\theta(\tau)$ with respect to θ. From Eq.(14), it can be seen that the direction in which parameter θ is optimized is such that it increases the probability $P_\theta(\tau)$ of trajectories τ with higher total returns $\mathcal{T}(\tau)$. Meanwhile, $\frac{\partial}{\partial \theta}\log P_\theta(\tau)$ could be partitioned into:

$$
\begin{aligned}
\frac{\partial}{\partial \theta}\log P_\theta(\tau) &= \frac{\partial}{\partial \theta}\log\left(P_\theta\left(s_0\right)\prod_{t=0}^{T-1}\pi_\theta\left(a_t \mid s_t\right)P_\theta\left(s_{t+1} \mid s_t, a_t\right)\right) \\
&= \frac{\partial}{\partial \theta}\left(\log P_\theta\left(s_0\right)+\sum_{t=0}^{T-1}\log\pi_\theta\left(a_t \mid s_t\right)+\sum_{t=0}^{T-1}\log P_\theta\left(s_{t+1} \mid s_t, a_t\right)\right) \\
&= \sum_{t=0}^{T-1}\frac{\partial}{\partial \theta}\log\pi_\theta\left(a_t \mid s_t\right).
\end{aligned}
\tag{15}
$$

The derivative with respect to θ of $P_\theta\left(s_0\right)$ and $P_\theta\left(s_{t+1} \mid s_t, a_t\right)$ are are equal to 0. It can be observed that $\frac{\partial}{\partial \theta}\log P_\theta(\tau)$ is independent of the state transition probabilities and only depends on the policy function. Therefore, the policy gradient $\frac{\partial \mathfrak{L}(\theta)}{\partial \theta}$ can be expressed as:

$$
\begin{aligned}
\frac{\partial \mathfrak{L}(\theta)}{\partial \theta} &= \mathbb{E}_{\tau\sim P_\theta(\tau)}\left[\left(\sum_{t=0}^{T-1}\frac{\partial}{\partial \theta}\log\pi_\theta\left(a_t \mid s_t\right)\right)\mathcal{T}(\tau)\right] \\
&= \mathop{\mathbb{E}}_{s_{1:T}\sim\mathcal{G}_\theta}\left[\sum_{t=1}^{T}\sum_{w_t\in\mathcal{V}}\nabla_\theta\log\pi_\theta(w_t|x_{1:t-1})\mathcal{T}_{\theta,\varphi}(s_{1:t},\mathcal{E})\right].
\end{aligned}
\tag{16}
$$

B.2 Proof of the MC rollout converge

Moreover, as the number of visits to / first visits tends to infinity, each / first-visit Monte Carlo rollout converges to $\pi_\theta\left(a \mid s\right)$. This is easily understood in the context of first-visit MC: each return is an independently and identically distributed estimate with the finite variance of $\mathcal{T}_{\theta,\varphi}(\mathrm{PW}_{1:t-1},\mathcal{E})$, and by the law of large numbers, the sequence of these estimates' means converges to the expected value. Each mean itself is an unbiased estimate, and the standard error of its error decreases as $1/\sqrt{n}$,

where n is the number of returns used to compute the mean. Assuming we conduct N trials and obtain N trajectories, denoted as $\tau^{(1)}, \tau^{(2)}, \ldots, \tau^{(N)}$, with corresponding total returns of $\mathcal{T}_{\theta,\varphi}(\tau^{(1)}), \mathcal{T}_{\theta,\varphi}(\tau^{(2)}), \ldots, \mathcal{T}_{\theta,\varphi}(\tau^{(N)})$, the total return can be approximated as follows:

$$\lim_{N\to\infty} \mathbb{P}\left(\left|\frac{1}{N}\sum_{n=1}^{N}\mathcal{T}_{\theta,\varphi}(\tau^{(n)}) - \mathcal{T}_{\theta,\varphi}(\mathrm{PW}_{1:t-1}, \mathcal{E})\right| > \varepsilon\right) = 0 \tag{17}$$

B.3 Notations

Table 9 summarizes the notations used in this paper, as well as the various categories of characters and the begin and end symbols used in this paper, which are ASCII 32-126.

Table 9: The notations used in this paper.

Notations	Descriptions
B_s E_s	Begin symbol and End symbol.
PW $\mathrm{PW}_{1:t}$	Password and partial password.
$\mathcal{L}_\mathrm{n}$ $\mathcal{D}_\mathrm{n}$ $\mathcal{S}_\mathrm{n}$	The letters, digits and special segment in length n.
$\mathcal{T}$ $\mathcal{E}$	Reward and expert.
$\mathbf{N}_1 \sim \mathbf{N}_{10}$	**N** stands for name usages, $\mathbf{N}_1$ for the usage of full name, $\mathbf{N}_2$ for the abbr. of the full name (e.g., `lw` from `leiwang`),$\cdots$.
$\mathbf{B}_1 \sim \mathbf{B}_5$	**B** stands for birthday usages, $\mathbf{B}_1$ for full birthday in YMD format (e.g., `19851106`), $\mathbf{B}_2$ for full birthday in MD, $\cdots$.
π (a,s) τ	π means password generation policy; (a,s) means guesser action and state pair; τ means a trajectory of password.

Table 10: Typical categories of characters used in this paper.

Character type	Symbols	Count
Lower letters	a b c d e f g h i j k l m n o p q r s t u w v x y z	26
Upper letters	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z	26
Digits	0 1 2 3 4 5 6 7 8 9	10
Special symbol	Space ! " # $ % & ' ( ) * + , – . / : ; 〈 = 〉 ? @ [\ ] ^ _ } { | } ~	33
Begin & End	B_s E_s	2
Total types	ASCII 32-126 and B_s E_s	97

Table 11: Comparison of three conditional guessing models.[†]

Experimental setup		PassRank-Mask	Pass-BERT [96]	CWAE [68]
Guessing scenario	Pivot			
CSDN → LinkedIn	*common*	85.57%	82.38%	95.44%
	uncommon	76.41%	72.86%	83.29%
	rare	66.50%	68.19%	84.84%
	super-rare	74.96%	60.29%	73.19%
CSDN → 000Webhost	*common*	79.14%	75.67%	76.87%
	uncommon	78.57%	74.26%	71.48%
	rare	40.02%	39.69%	38.94%
	super-rare	18.30%	24.81%	15.02%
Wishbone → Dodonew	*common*	88.21%	86.42%	79.43%
	uncommon	71.22%	70.29%	68.00%
	rare	56.36%	54.34%	49.73%
	super-rare	42.56%	48.06%	33.15%
Wishbone → Taobao	*common*	89.60%	87.52%	85.14%
	uncommon	84.03%	79.26%	80.76%
	rare	66.02%	67.08%	65.33%
	super-rare	55.42%	36.66%	39.28%
#A1: Engmix → LinkedIn	*common*	91.41%	88.94%	79.60%
	uncommon	89.32%	81.30%	80.49%
	rare	85.86%	83.28%	82.33%
	super-rare	81.09%	78. 67%	76.85%
#A2: Enmix → Taobao	*common*	89.14%	85.53%	88.15%
	uncommon	73.09%	70.26%	71.95%
	rare	75.86%	69.58%	71.90%
	super-rare	73.09%	70.59%	68.42%
#B1: Chmix → Rockyou	*common*	92.12%	91.80%	89.13%
	uncommon	89.52%	86.72%	83.19%
	rare	85.17%	81.16%	80.57%
	super-rare	83.46%	74.72%	50.19%
#B2: Chmix → Taobao	*common*	98.10%	96.72%	95.53%
	uncommon	93.47%	91.36%	90.65%
	rare	89.13%	87.30%	85.41%
	super-rare	84.19%	81.23%	80.58%
#C1: Mix → LinkedIn	*common*	99.81%	97.29%	95.39%
	uncommon	94.01%	91.22%	90.94%
	rare	91.72%	87.32%	90.37%
	super-rare	89.85%	86.37%	85.68%
#C2: Mix → Dodonew	*common*	98.59%	97.34%	94.71%
	uncommon	94.74%	95.99%	91.31%
	rare	91.35%	90.92%	88.87%
	super-rare	80.35%	60.41%	72.32%

[†]A value with dark gray (resp. light gray) represents the highest one (resp. 2nd one).

C Supplementary experimental results

This section focuses on targeted guessing based on personal information (PII) in Table 12, along with supplementary experiments on conditional password guessing in Table 11, to demonstrate the remarkable effectiveness of our model.

In targeted password guessing, models specialized in guessing passwords with personally identifiable information (PII) are assessed including TarGuess-I [87], Targeted-Markov [85], and RFGuess-PII [89] employ a random forest for the same purpose. Finally, in conditional password guessing, two models are investigated, and we explore CWAE [68] and PassBERT using fine-tuning a pre-trained model with medium configurations the same as Xu *et al.* [96].

This comprehensive analysis offers valuable insights into password security techniques, significantly advancing our understanding and contributing to the enhancement of password security practices and robust cybersecurity measures.

Table 12: Comparison of the cracking rate of four different attack methods*

Experimental setup		PassRank-PII	RFGuess-PII [89]	Targuess-I [87]	TarMarkov-3order [85]	TarMarkov-4order [85]
Attacking scenario	Number of guesses					
#1: 1M PII-000Webhost→PII-000Webhost_rest	10	**8.36%**	5.78%	8.12%	1.95%	1.98%
	10^5	13.99%	**18.18%**	14.32%	13.08%	13.53%
	10^8	**23.38%**	23.22%	20.59%	22.16%	22.20%
	10^{14}	**73.06%**	62.52%	36.43%	51.46%	53.32%
#2: 1M PII-CSDN → PII-CSDN_rest	10	14.36%	**24.20%**	14.55%	16.47%	15.55%
	10^5	**34.51%**	33.45%	34.11%	34.08%	34.48%
	10^8	**58.69%**	58.06%	34.08%	47.18%	48.92%
	10^{14}	**96.55%**	91.44%	43.67%	77.29%	79.99%
#3: 1M PII-12306 → PII-12306_rest	10	12.53%	**16.32%**	11.28%	8.53%	8.54%
	10^5	**33.29%**	33.21%	29.88%	31.70%	31.56%
	10^8	57.92%	**59.97%**	34.11%	49.82%	46.07%
	10^{14}	**96.68%**	82.40%	44.86%	88.01%	83.43%
#4: 1M PII-Rootkit → PII-Rootkit_rest	10	5.57%	**7.24%**	6.18%	4.91%	4.92%
	10^5	**33.22%**	23.21%	22.05%	20.21%	20.21%
	10^8	**42.78%**	42.54%	28.66%	29.98%	30.01%
	10^{14}	**89.37%**	82.40%	31.34%	69.89%	70.12%
#5: 1M PII-ClixSense → PII-ClixSense_rest	10	8.36%	7.14%	**9.14%**	8.16%	8.11%
	10^5	**35.16%**	31.79%	27.55%	29.23%	30.25%
	10^8	**60.73%**	57.21%	31.63%	51.32%	52.43%
	10^{14}	**96.36%**	86.16%	56.93%	79.90%	84.21%
#6: PII-Mixed†: 90% → 10%	10	**18.07%**	15.36%	17.13%	16.97%	17.73%
	10^5	**31.91%**	29.50%	26.32%	27.89%	28.66%
	10^8	67.86%	**68.32%**	66.62%	63.69%	61.88%
	10^{14}	**99.28%**	96.36%	93.19%	91.82%	92.96%
#7: PII-Mixed: 70% → 30%	10	**16.55%**	15.36%	14.1%	13.94%	14.72%
	10^5	28.70%	27.60%	**29.31%**	26.63%	27.47%
	10^8	**62.67%**	60.17%	61.43%	57.58%	58.85%
	10^{14}	**93.60%**	91.25%	65.66%	88.23%	89.15%
#8: PII-Mixed: 50% → 50%	10	**16.17%**	15.16%	14.32%	12.86%	13.48%
	10^5	27.41%	26.78%	**28.76%**	26.90%	27.57%
	10^8	**57.16%**	56.44%	58.91%	49.77%	50.87%
	10^{14}	**91.73%**	89.23%	61.27%	85.50%	86.79%
#3A: $\frac{1}{2}$ PII-12306 → PII-CSDN	10	**11.88%**	11.28%	12.66%	10.28 %	10.30%
	10^5	**24.69%**	23.56%	22.13%	20.78%	21.53%
	10^8	**47.19%**	45.48%	46.16%	41.36%	40.19%
	10^{14}	**81.50%**	78.13%	62.64%	75.66%	76.96%
#3B: $\frac{1}{4}$ PII-12306 → PII-CSDN	10	**10.31%**	9.73%	9.87%	8.52%	7.61%
	10^5	**18.68%**	17.35%	18.39%	17.10%	15.39%
	10^8	**41.92%**	38.34%	40.32%	38.61%	36.53%
	10^{14}	**72.36%**	69.63%	58.98%	61.94%	60.73%
#3C: $\frac{1}{8}$ PII-12306 → PII-CSDN	10	**7.12%**	7.46%	7.65%	5.27%	6.71%
	10^5	**14.66%**	14.58%	13.79%	12.13%	12.78%
	10^8	**26.77%**	26.16%	24.11%	23.33%	22.66%
	10^{14}	**58.02%**	56.21%	49.66%	55.44%	51.14%
#3A: $\frac{1}{2}$ PII-ClixSense → PII-000Webhost	10	**9.47%**	9.13%	8.56%	8.18%	7.93%
	10^5	**18.01%**	17.49%	16.60%	16.30%	15.85%
	10^8	**23.89%**	23.12%	24.45%	22.40%	21.75%
	10^{14}	**62.50%**	58.88%	53.56%	55.05%	59.62%
#3B: $\frac{1}{4}$ PII-ClixSense → PII-000Webhost	10	8.51%	7.28%	**8.98%**	6.24%	7.18%
	10^5	**16.07%**	15.65%	15.39%	12.38%	14.90%
	10^8	**23.92%**	22.48%	22.59%	20.70%	23.07%
	10^{14}	**58.36%**	55.67%	52.09%	50.01%	51.48%
#3C: $\frac{1}{8}$ PII-ClixSense → PII-000Webhost	10	**8.48%**	7.39%	8.12%	8.01%	8.31%
	10^5	**14.66%**	13.76%	12.91%	13.20%	14.63%
	10^8	**25.21%**	24.94%	24.16%	21.03%	23.38%
	10^{14}	**57.02%**	55.20%	48.93%	52.23%	52.54%

* We considered 14 guess experiments of cross-site scenarios. A **bold** value in each row means that it is the highest one among all five methods.

† We mix three English datasets (i.e., 000Webbhost, Rootkit, and ClixSense) and three Chinese datasets (i.e., CSDN, 12306, and Dodonew) and use 90%, 70% and 50% of them for training and the rest 10%, 30% and 50% for testing. Further elucidate the stability of our approach in the case of a larger training dataset.